

ZeroMerma Operational Database

Comprehensive Schema Design for Multi-Branch Bakery Management

Data Engineering — ZeroMerma

August 22, 2025

Contents

1	Purpose, Scope, and Target Platform	2
2	Design Standards and Conventions	3
3	Administration Module	4
3.1	Branch (Sucursal)	4
3.2	Role (Rol)	8
3.3	User (Empleado/Usuario)	10
4	Catalog Module	14
4.1	Product Category (CategoríaProducto)	14
4.2	Product (Producto)	16
4.3	Recipe and Recipe Item (Receta y RecetaDetalle)	22
5	Inventory Module	27
5.1	Inventory (Existencias por sucursal)	27
5.2	Inventory Movement (MovimientoInventario)	31
6	Sales Module (POS)	36
6.1	Customer (Cliente)	37
6.2	Cash Session (Apertura/Cierre de Caja)	39
6.3	Sale and Sale Item (Venta y Detalle)	43
6.4	Sale Payment (Pago de Venta)	47
7	Purchases Module	49
7.1	Supplier (Proveedor)	50
7.2	Purchase and Purchase Item (Compra y Detalle)	51

8	Production Module	56
8.1	Production (Lote de Producción)	56
8.2	Production Detail (Uso Real de Insumos) – optional	61
9	Transfers Module	61
9.1	Transfer (Traslado de stock)	62
9.2	Transfer Item (Detalle de Transferencia)	64
10	Forecasting Module (Pronósticos de Demanda)	66
10.1	Sales Forecast (Pronóstico de Venta por IA)	66
11	Telemetry Module	69
11.1	Sensor (Dispositivo IoT)	70
11.2	Sensor Reading (Lectura de Sensor)	71
A	Glossary of Terms	72
B	Integrity Constraints and Validation Rules	74
C	Indexing and Partitioning Strategy	78
D	Sample SQL DDL Fragments	82

1 Purpose, Scope, and Target Platform

This document is a complete and self-contained specification of the **ZeroMerma operational database**. It serves as a technical manual for building and understanding the database, and doubles as a tutorial for its use in a multi-branch bakery management system . The design is implemented on a relational platform (PostgreSQL 14+ recommended), though it can be adapted to other SQL databases with minimal changes . The system covers all functional areas of the bakery operation: Point-of-Sale (sales), Inventory management, Purchases (ordering supplies), Production (batch baking records), Inter-branch stock Transfers, Cash register Sessions, Customer and Supplier management, as well as Telemetry/IoT data collection and AI-based demand Forecasting . The goal is to achieve full **traceability** of operations and data-driven decision support across these modules: every table plays a specific role and links with others to maintain a coherent, auditable record of what happens in the business .

In what follows, each module of the schema is detailed table-by-table. For each table, we present its structure (fields, data types, nullability, constraints, defaults), an explanation of each field's purpose and any integrity or validation rules, the table's relationships to others, and example scenarios of real-world usage. Redundant content from the source documents has been omitted, but all unique details are retained. (Entity-Relationship diagrams are omitted as per specification.) This comprehensive design adheres to normalization (3NF) for core transactional data, while ensuring performance with selective denormalizations (e.g. storing totals) and appropriate indexing. Strong referential integrity and auditability are enforced through foreign keys and timestamp/user audit fields on each table. All business workflows should be executed as ACID transactions in the application layer to keep data consistent with this schema.

2 Design Standards and Conventions

The database design follows these standards and conventions to ensure consistency and integrity:

- **Naming:** All identifiers use `snake_case` in English (Spanish equivalents are noted in the glossary). Primary key fields are named `id.<entity>` (e.g. `id.branch`), and foreign keys use the pattern `<entity>_id` referring to the referenced table (e.g. `branch_id` in related tables).
- **Data Types:** Numeric types are chosen for precise values like money and quantities. For example, monetary amounts use `NUMERIC(12,2)` and inventory weights/volumes use `NUMERIC(12,3)` or a similar scale. Temporal values are stored as timestamp with time zone (PostgreSQL `TIMESTAMPTZ`) in UTC, with formatting to local time handled in the application layer. Strings use `VARCHAR` with length limits as needed (or `TEXT` for long descriptions). Boolean flags (`BOOLEAN` or bit) denote binary states.
- **Integrity Policies:** The schema prefers *soft deletes* for master records instead of physical deletion. Many tables include an `is_active` flag to deactivate records (e.g. products, customers) rather than delete them, preserving history. All foreign keys enforce referential integrity (typically using `ON DELETE RESTRICT` for references to critical master data, so you cannot delete a referenced parent record, and `ON DELETE CASCADE` for subordinate detail records that should be removed with

their parent). In some cases `ON DELETE SET NULL` is used if nullifying a reference is acceptable when the parent is removed (e.g. an optional relationship to a user). These constraints prevent orphan records and ensure consistency (e.g. the DB will not allow a sale referencing a non-existent branch). All `NOT NULL` constraints are applied to fields that are mandatory for a valid record. Fields that must be unique (such as codes or names where applicable) have `UNIQUE` constraints. Check constraints are used to enforce basic validations at the database level (e.g. non-negative quantities).

- **Time & Money Handling:** Timestamps are recorded in UTC and automatically set on insert for creation time (and on update for modification time, where tracked). Monetary values are stored in the company's base currency (e.g. MXN) in a single currency for simplicity (a separate currency table could be introduced for multi-currency support). All money and quantity fields are non-floating type to avoid rounding errors (as noted, `NUMERIC` with appropriate scale is used).

With these conventions in place, the schema ensures clarity (via naming standards), reliable data integrity (via constraints and transaction rules), and auditability (via timestamp and user attributions).

3 Administration Module

The Administration module covers global entities that are used throughout the system: branches (physical bakery locations), user accounts (employees), and roles (user privilege profiles). These tables define the organizational structure and who can access the system.

3.1 Branch (Sucursal)

The **branch** table stores each physical store or bakery location in the company. Key fields include the branch's name and contact information, an active/inactive flag, and audit fields for creation. New branches are added here and linked to other modules (inventory, sales, etc.). The structure is:

Field	Data Type	Null?	Key / Constraints	Con-	Default
<code>id_branch</code>	BIGINT	No	PK (primary key)		(auto)
<code>name</code>	VARCHAR(100)	No	UNIQUE		–
<code>address</code>	VARCHAR(255)	Yes			–
<code>phone</code>	VARCHAR(20)	Yes			–
<code>email</code>	VARCHAR(100)	Yes			–
<code>city</code>	VARCHAR(50)	Yes			–
<code>is_active</code>	BOOLEAN	No			TRUE
<code>created_at</code>	TIMESTAMPTZ	No			NOW()
<code>created_by</code>	BIGINT	No	FK user(id_user)	→	–
<code>updated_at</code>	TIMESTAMPTZ	Yes			–
<code>updated_by</code>	BIGINT	Yes	FK user(id_user)	→	–

****Field meanings:****

- **id_branch:** Unique branch identifier (surrogate primary key). This is referenced as a foreign key by many other tables to tie records to a specific location . It is auto-generated (auto-increment).
- **name:** The branch’s name (e.g. "Downtown Bakery"). This must be unique to clearly distinguish branches . It cannot be null (every branch needs a name). Validation rules (not enforced by DB schema but at application) might restrict format to alphanumeric and spaces and limit length to 100 characters.
- **address, phone, email, city:** Contact and location details of the branch. These fields are optional (nullable) and for information purposes. If provided, they store the branch’s physical address, phone number, email, and city/region respectively . They help in communications or regional reporting but are not mandatory for operation.
- **is_active:** Boolean flag indicating if the branch is operational (1 or TRUE) or deactivated/closed (0 or FALSE). Rather than deleting a branch that closes, setting it inactive preserves its history while excluding it from new transactions . This field is not nullable and defaults to TRUE (active) for new branches .
- **created_at:** Timestamp when the branch record was created in the system. This is set automatically at insertion. It provides an audit

trail of when each branch was added .

- **created_by**: Reference to the **user** who created the branch record (the admin or user ID who performed the registration). This is a foreign key to the **user** table and is required, ensuring we record who created each branch . This supports accountability and traceability of data changes.
- **updated_at**: Timestamp of the last update to the branch record. This is nullable (it remains NULL until an update occurs). It is for auditing changes.
- **updated_by**: Foreign key to **user**, indicating who last updated the branch record. Nullable; if no updates have been made or if the updating user was later deleted, it can be NULL. If a user is deleted, we set this to NULL (on delete set null) to avoid foreign key issues while retaining the fact the branch exists .

****Relationships:**** One branch can be associated with many records in other tables:

- **Branch–User (1–N)**: A branch employs many users (staff), while each user is primarily assigned to one branch . This is implemented via a foreign key **branch_id** in the **user** table linking to **branch**. This association allows filtering users by branch (e.g. list all employees of Branch X) and ensures that operational records (sales, etc.) made by a user can be traced back to that user’s branch.
- **Branch–Inventory (1–N)**: Each branch maintains its own stock inventory. The **inventory** table (see Inventory module) has one record per product per branch, meaning a branch will have many inventory records (for different products) . This relation tracks stock levels of each product at each location.
- **Branch–Sale (1–N)**: Sales transactions are linked to the branch where the sale occurred. The **sale** table includes a **branch_id** FK; a branch will have many sale records . This allows computation of sales per branch, performance comparisons, etc.
- **Branch–Purchase (1–N)**: Supply orders (purchases) may be placed by or delivered to specific branches (especially if each branch manages its own supply). The **purchase** table records which branch the order is for; one branch can have many purchases recorded .

- **Branch–Production (1–N):** Production batches of baked goods happen at a branch’s kitchen. The **production** records carry a branch reference for where the batch was produced. A branch will have many production entries over time .
- **Branch–Transfer (1–N, in two roles):** For stock transfers between branches, each transfer involves two branch relationships: an origin branch and a destination branch. In the **transfer** table, there are two FKs (**origin_branch_id** and **dest_branch_id**), each referencing **branch**. Thus a given branch can appear as origin in many transfers (outgoing) and as destination in many transfers (incoming) .
- **Branch–Sensor (1–N):** If the system includes IoT sensors for telemetry, sensors are installed at specific branches. The **sensor** table links each device to the branch where it is deployed. One branch can have many sensors (e.g. ovens, ambient sensors, counters) .
- Additionally, other operational tables such as cash **session** (register open/close records) also reference branch, since a cash register session belongs to a particular store location .

These foreign keys enforce that a record in any of those tables cannot exist without a valid branch (you cannot record a sale, inventory, etc. for a non-existent branch). The DBMS will prevent inserting child records if the parent branch does not exist, and will prevent deleting a branch if related records still exist, ensuring referential integrity .

****Usage Scenarios:****

1. **Opening a New Branch:** Suppose the bakery opens a new location. An administrator uses the admin interface to create a new branch record. On save, a new row is inserted into **branch** with a fresh **id_branch**; the **created_by** is set to the admin’s user ID, and **created_at** to the current timestamp . Immediately after, the admin can assign employees to this branch (either by creating new **user** records or updating existing users to set their **branch_id** to the new branch) . Also, an initial inventory record for each product can be initialized for the new branch (with stock quantity 0 until stock arrives) . Once operational, all transactions at that branch (sales, production, etc.) will reference its ID, allowing separate tracking of that branch’s performance .
2. **Deactivating a Branch:** If a branch shuts down or is temporarily closed, the admin can set **is_active** = 0 for that branch instead of

deleting it. This way, the branch no longer appears in selection lists for new transactions, but all historical data (sales, etc.) remain linked to it. For example, if "Downtown" branch closes, mark it inactive. Historical sales still reference Downtown, preserving analytics. The DB will prevent deletion of the branch while child records exist, so deactivation is the safe approach. Reactivation (set `is_active = 1`) is possible if it reopens.

3.2 Role (Rol)

The **role** table defines the various user roles or access profiles within the system (e.g. Admin, Cashier, Baker). It acts as a lookup for user accounts, allowing role-based permissions. The schema is simple:

Field	Data Type	Null?	Key / Constraints	Con-	Default
<code>id_role</code>	BIGINT	No	PK		(auto)
<code>code</code>	VARCHAR(40)	No	UNIQUE		—
<code>name</code>	VARCHAR(100)	No			—
<code>description</code>	TEXT	Yes			—
<code>is_active</code>	BOOLEAN	No			TRUE

****Field meanings:****

- **id_role:** Primary key identity for the role.
- **code:** A short code or identifier for the role (e.g. "ADMIN", "CASHIER", "BAKER"). This is constrained unique and not null. It provides a stable, concise identifier that can be used in configuration or code (for example, to toggle features by role code) .
- **name:** A human-friendly name of the role, for display (e.g. "Administrator", "Cashier"). This may be somewhat descriptive. It is required (not null), and typically one would ensure it's unique as well (though uniqueness is enforced by the code, the name is not strictly required to be unique if code is unique, but usually they would be distinct).
- **description:** A longer description of the role's purpose or permissions. Optional (can be NULL). For example, description for "Admin" might read "Full access to all modules".
- **is_active:** Boolean flag indicating if this role is active/available for assignment. Defaults to TRUE (active). This allows disabling a role

(set to FALSE) if it's no longer used or should not be assigned, without deleting it. Like other master data, roles are generally not deleted if already in use by users; instead we mark inactive to prevent new use while preserving existing user-role links.

****Relationships:****

- **Role–User (1–N):** Each user account is assigned one role (see `user.role_id` foreign key), and a given role can be held by many users. For example, dozens of users may have role "CASHIER". This relationship is many users to one role. By using a role table, we avoid repeating role names in every user record and can change permission logic in one place.

Roles are relatively independent; other tables generally do not reference roles directly except through users. (In a more advanced system, roles could link to permission sets, but that's outside our scope. Here, roles are primarily for classification of users.)

****Usage Scenarios:****

- **Creating a New Role:** If the organization grows and needs a new role, for example "Production Chief", an administrator would insert a new record into the role catalog with `code = 'PROD.CHIEF'` and name "Jefe de Producción". Once added, this role can be assigned to users. This approach centralizes role definitions so that adding a role in this table makes it immediately available throughout the application (e.g. in drop-downs for user roles).
- **Role-Based Access Control:** The application uses the user's role to determine what functions they can perform or see. For instance, if a user's role code is "CASHIER", the POS screen is enabled for them but configuration screens are hidden; if role is "ADMIN", all menus are accessible. This is implemented in application logic but relies on the role assigned to the user from this table. By changing a user's role (e.g. promoting a cashier to an admin), their permissions change accordingly without altering individual records.
- **Reporting by Role:** One could generate a quick report of how many users are in each role or list users by role. For example, counting users with role "Baker" vs "Cashier" can indicate staffing levels. Also, if the system tracks user actions (sales, production entries, etc.), one could filter those by role via joining through the user table (e.g. find all sales made by users with role "CASHIER").

3.3 User (Empleado/Usuario)

The **user** table holds all employee login accounts for system access, including their personal info, credentials, role, and status. It links the human operators to their actions in the system (sales made, inventory adjustments, etc.) and provides authentication data. The structure:

Field	Data Type	Null?	Key Con- straints	/	Default
id.user	BIGINT	No	PK		(auto)
first_name	VARCHAR(50)	No			–
last_name	VARCHAR(50)	No			–
email	VARCHAR(120)	No	UNIQUE		–
password_hash	VARCHAR(255)	No			–
role_id	BIGINT	No	FK	→	– role(id.role)
branch_id	BIGINT	Yes	FK	→	NULL branch(id.branch)
is_active	BOOLEAN	No			TRUE
hire_date	DATE	Yes			–
termination_date	DATE	Yes			–
created_at	TIMESTAMP TZ	No			NOW()
created_by	BIGINT	No	FK	→	– user(id.user)
updated_at	TIMESTAMP TZ	Yes			–
updated_by	BIGINT	Yes	FK	→	– user(id.user)

Field meanings:

- **id.user**: Primary key for the user (employee). Used to reference the user in other tables (e.g. recording which user performed an action) . Typically an auto-incrementing ID.
- **first_name, last_name**: The user's personal name. Both are required (not null). We separate first and last name for clarity; together they form the user's full name as seen in reports or the UI . (Alternatively, a single full name field could be used, but here it's split.)
- **email**: The user's email address. This is required and must be unique across all users . The email often serves as the login identifier. It's

also used for communication (password resets, notifications). The application should validate it as a proper email format upon entry.

- **password_hash**: The hashed password for authentication (not the plain text password). This is required (users cannot be created without credentials). Length 255 is chosen to accommodate hashes from algorithms like bcrypt or SHA-256 with salt . The system never stores plain passwords, only secure hashes.
- **role_id**: Foreign key to the **role** table, indicating the user's role or job function (e.g. which set of permissions they have) . This is mandatory; every user must have a role assigned, as it defines their access level. The referential integrity ensures the role must exist in the role catalog.
- **branch_id**: Foreign key to **branch**, indicating the branch at which this user primarily works. This can be NULL if the user is not tied to a single branch (e.g. a regional manager or IT support who moves between branches) . However, for most store staff (cashiers, bakers), this is filled in. If not null, the branch ID must exist in the branch table (the FK ensures that). This association is used to, for example, filter data entry or sales by the user's branch.
- **is_active**: Boolean indicating if the user account is active (TRUE) or deactivated (FALSE) . An employee who left the company or is suspended can be marked inactive to prevent login and remove from assignment lists, while preserving their past records (sales they made, etc.). This field defaults to TRUE when a user is created.
- **hire_date**: The date the employee was hired (or joined the company). Optional (NULL if not tracked). Useful for HR or calculating tenure, but not required for system functionality .
- **termination_date**: The date the employee left the company, if applicable. NULL if the person is still employed. This helps identify if an **is_active = FALSE** user left on a particular date, etc. .
- **created_at**: Timestamp when this user record was created. Set at insertion time automatically. Not null (every record will have a creation time) .
- **created_by**: Foreign key to **user** (self-referential). This records which user (typically an admin) created this user account . It provides traceability for user onboarding. This cannot be null – one must always

record who created a user. If the creator user is later deleted or made inactive, we may use ON DELETE SET NULL or PROTECT; in our design, we protect this relationship (the system may restrict deleting a user who is referenced as creator of others, or use a special admin account for creation).

- **updated_at, updated_by:** Timestamp of last update and the user who last updated the user record, respectively. These are analogous to the created_at/created_by fields, but for modifications. They are optional (null if never updated, or if the updater account was removed we allow null). These fields, along with created_at/created_by, form the audit trail for the user records.

****Relationships:****

- **User–Role (N–1):** Many users can share the same role (e.g. dozens of users might have the "Cashier" role). Implemented via the `role_id` foreign key in `user`. This allows grouping users by role, changing permissions by editing role data, etc.
- **User–Branch (N–1):** Many users belong to one branch (as primary workplace). This is `branch_id` in `user`. It enables filtering users by branch, but also logically associates their actions (sales, etc.) with that branch. For example, when a cashier records a sale, we know their branch too (both via the sale's branch and the user's branch).
- **User–(Operational Records):** The user's ID appears as a foreign key in numerous operational tables to record who performed an action:
 - In `sale`, a field like `cashier_id` links to the user who handled the sale. A user can be responsible for many sale records, but each sale has one cashier.
 - In `cash_session`, fields link to the user who opened or closed the register session (often a single user opens and closes their own session).
 - In `production`, the `responsible_id` (and possibly `approved_by`) link to users who carried out or supervised a production batch. One user can oversee many productions.
 - In `purchase`, a `created_by` (or similar `ordered_by`) would reference the user who placed the order or logged the purchase receipt (inventory manager). One user may log many purchases.

- In `transfer`, the `enviado_por` and `recibido_por` fields reference the users who sent and received a stock transfer . A user might be involved in many transfers over time (sending or receiving).
- In `inventory_movement`, `user_id` references who recorded the movement (e.g. the staff performing a stock adjustment) .

These relationships are one-to-many: one user has many associated sales, sessions, productions, etc., whereas each individual record is linked to a single user (or two, in the case of transfers) responsible for it. This web of relationships ensures every important transaction can be traced to a user, providing accountability .

- **User–User (1–N self-relationship):** The `created_by` (and `updated_by`) in the user table create a recursive relationship: a user record can be "created by" another user (often an admin) . Over time, one admin might create many user accounts, but each user account has only one recorded creator. This is a 1–N self-referential link, helpful for auditing user management. We typically protect or nullify this reference if the creator account is deleted, to avoid losing the trail.

****Usage Scenarios:****

- **Onboarding a New Employee:** When a new employee is hired, an administrator creates a new `user` entry for them. For example, a cashier "Juan Pérez" is added with `role = Cashier` and assigned to the "Downtown" branch . The system sends Juan's login credentials to the provided email. When Juan first logs into the POS, the system verifies his email and password against the `user` table (matching the hash) and checks that `is_active = 1` . If valid, he gains access. His branch and role (also stored in this table) determine what interface he sees and what data he can access.
- **Authentication and Permissions:** Each login attempt uses the `email` (unique identifier) and the entered password (hashed to compare to `password_hash`) to authenticate . Once logged in, the application checks the user's role via `role_id`. For instance, if Juan is later promoted to manager, his `role_id` is updated to the Manager role. Now the system will automatically grant him the additional permissions associated with that role (like approving discounts or viewing reports) without needing a new account.

- **Tracking Actions by User:** All transactions record the user who performed them. For example, every sale has a `cashier_id`. If there is an issue with a sale (like a refund or discrepancy), managers can see which cashier made the sale. Similarly, an inventory adjustment record has the `user_id` of who entered it, so accountability is maintained for stock changes. If a user leaves the company (`termination_date` set and `is_active` = 0), their ID still remains on historical records, preserving the audit trail. The referential integrity ensures that even if the user account is inactive or even deleted, their ID remains on those operations (we typically would not delete a user with associated records; if we did, we might set those references to NULL or use a placeholder "deleted user" account).
- **Deactivating a User Account:** If an employee resigns or is let go, their user account is marked inactive (set `is_active` = 0). They can no longer log in, but all their past actions (sales, etc.) remain attributed to them. We do not delete the user because that would remove the link on historical records. For example, sales made by "Juan" will still show his user ID (or name in reports) even after Juan is deactivated, which is important for historical integrity. If privacy regulations require removing personal data, one might anonymize the name but keep the record.

4 Catalog Module

The Catalog module contains master data about products and related reference information that is used across the system. This includes product categories (to group products), the products themselves (both finished goods and raw materials), and recipes that define how products are produced from ingredients. These are relatively static data that define what the bakery sells and uses.

4.1 Product Category (CategoríaProducto)

The `category` table defines groupings for products (e.g. "Breads", "Pastries", "Beverages", "Ingredients"). Categories help organize products for reporting, menu grouping, and potentially applying common rules (like tax rates). The table structure:

Field	Data Type	Null?	Key / Constraints	Default
<code>id_category</code>	BIGINT	No	PK	(auto)
<code>name</code>	VARCHAR(80)	No	UNIQUE	–
<code>description</code>	VARCHAR(255)	Yes		–
<code>is_active</code>	BOOLEAN	No		TRUE

****Field meanings:****

- **`id_category`:** Primary key for the category.
- **`name`:** The category name (e.g. "Savory Breads", "Sweet Pastries", "Beverages", "Ingredients"). Required and unique . This is what is displayed in product lists or POS menus as the grouping label. Uniqueness ensures no two categories have the same name, avoiding confusion.
- **`description`:** An optional longer description of the category's contents or purpose (e.g. "Bread products with savory fillings"). Mostly for internal documentation.
- **`is_active`:** Boolean flag for soft deletion of categories. Defaults TRUE (active). If a category is no longer used, instead of deleting (which might break links to existing products), it can be marked inactive (FALSE) so it's hidden from selection going forward but remains for historical data integrity.

****Relationships:****

- **Category–Product (1–N):** A category can have many products. Each product has a `category_id` FK linking to this table . For example, category "Beverages" may have products "Coffee", "Tea", etc. This relationship is the main purpose of categories: to categorize products for filtering and reporting. The foreign key ensures referential integrity; you cannot assign a product to a non-existent category, and deleting a category is prevented if products are still assigned to it. In practice, rather than deleting categories with products, one would mark them inactive or reassign the products first .

****Usage Scenarios:****

- **Initial Category Setup:** When implementing the system, the admin sets up several base categories. For example: "Bread - Savory", "Bread - Sweet", "Cakes", "Drinks", "Ingredients" . This helps organize the

product catalog. Later, when adding new products, the user must choose one of these categories.

- **Using Categories in POS Menu:** The POS interface can list categories as top-level buttons (e.g. user taps "Pastries" then sees all pastry products). Because every product is linked to a category, the system can group them easily. If the bakery introduces a new category (say "Seasonal Specials"), adding it here and assigning products to it will automatically reflect in the POS grouping.
- **Reporting by Category:** Managers can get reports like "sales by category". For instance, total sales of "Beverages" vs "Breads". This is done by grouping sales or products by the category foreign key. The category table enables such analysis without hardcoding category names on each product.
- **Retiring a Category:** Suppose "Cakes" category is to be phased out (maybe split into subcategories). Instead of deleting it (which would be blocked if products exist), the admin marks `is_active = 0` for "Cakes". The application then stops showing "Cakes" as an option for new products or filtering, but historical data remains intact. Any products still in that category could either be recategorized or also marked inactive. The referential integrity ensures no product is left pointing to a missing category.

4.2 Product (Producto)

The **product** table is central to the system: it defines every item that can be sold or used as an ingredient. This includes finished goods (like a loaf of bread or a pastry) and raw materials (flour, sugar, etc.), distinguished by a flag. Each product has pricing and inventory-related attributes. The structure:

Field	Data Type	Null?	Key / Con- straints	Default
id_product	BIGINT	No	PK	(auto)
sku	VARCHAR(40)	No	UNIQUE	–
name	VARCHAR(120)	No		–
category_id	BIGINT	No	FK → category(id.category)	–
is_input	BOOLEAN	No		FALSE
measure_unit	VARCHAR(20)	No		–
sale_price	NUMERIC(12,2)	Yes		NULL
standard_cost	NUMERIC(12,4)	Yes		NULL
min_stock	NUMERIC(12,3)	Yes		NULL
recipe_id	BIGINT	Yes	FK → recipe(id.recipe)	NULL
is_active	BOOLEAN	No		TRUE
created_at	TIMESTAMPTZ	No		NOW()
created_by	BIGINT	No	FK → user(id.user)	–
updated_at	TIMESTAMPTZ	Yes		–
updated_by	BIGINT	Yes	FK → user(id.user)	–

Field meanings:

- **id_product**: Primary key for the product.
- **sku**: Stock Keeping Unit – an internal code used to uniquely identify the product (could be numeric or alphanumeric). This is not meant for humans but for inventory systems, barcodes, etc. It must be unique and is not null . If the bakery uses barcodes, the SKU might correspond to a barcode value.
- **name**: The product name (for display and identification, e.g. "Glazed Donut", "Whole Wheat Flour 50kg"). Required. This should be descriptive enough for selection in the POS or inventory lists . If the product is a raw material, the name might include packaging (as in "Flour 50kg sack").
- **category_id**: Foreign key to the product category. Required for all products . This places the product in one of the categories defined in the previous table. It's used for organization and possibly business

logic (like applying certain tax or handling all "Ingredient" category items differently). Integrity is enforced so the category must exist.

- **is_input**: Boolean flag indicating the type of product: **TRUE (1)** if this product is an *input*/raw material (insumo) used in production and typically not sold directly to customers, **FALSE (0)** if it is a finished good sold to customers . This distinction is crucial. A product marked as input might be flour, sugar, etc. It will not appear on the POS sales screen (not for direct sale), and can appear in recipes as an ingredient. A product with **is_input = FALSE** is a sellable item; it can have a sale price and optionally a recipe (if it's produced in-house) . This field is not nullable (must be explicitly set to false for sellable products). We default it to FALSE (assuming most items are finished goods unless specified as inputs).
- **measure_unit**: The unit of measure for this product. Example values: "pz" (piece), "kg", "L", "pack" etc. Not null . This field standardizes how the quantity is interpreted (especially important for inputs which might be stored in kg, etc.). We do not normalize units into a separate table here (though we could); instead we rely on consistent strings. This should align with how stock and recipes are measured for this item.
- **sale_price**: The retail price of the product (per unit) for sellable products. Optional (NULL or 0 if not applicable). For products that are sold, this is how much one unit costs to the customer (e.g. \$10.00 MXN for a concha bread) . For input items (raw materials not sold individually), this may be left NULL or zero since they aren't sold to customers . The system should enforce logical rules: a product that is sellable (**is_input = FALSE**) should ideally have a **sale_price** set, whereas an input can have it null. Negative prices are disallowed by business logic (a check could ensure price ≥ 0).
- **standard_cost**: The standard cost of the product per unit. This is used for inventory valuation or cost of goods calculations. For a finished product, it might be an estimated production cost per unit; for an input, it could be the average purchase cost per unit . Optional field. It helps in computing profit margins, etc. Must not be negative. Typically filled for inputs with the latest cost, and for finished goods with the sum of ingredient costs or a target cost .

- **min_stock:** The minimum desired stock level (reorder point) for this product across branches. Optional. If set, it's a guideline that when total stock falls below this, replenishment is needed. In a multi-branch context, this could be interpreted as a general threshold or separately per branch (in fact, our design also includes per-branch reorder points in the inventory table). This field can serve as a default reorder threshold that can be overridden per branch. It's typically more relevant for inputs (ingredients) where you want to keep at least X in stock, or critical finished goods. If not used, it can be NULL.
- **recipe_id:** (Optional) Foreign key to a **recipe** table entry that defines how this product is produced. This is used for *finished goods that are made from other ingredients*. If the product has an associated recipe, that recipe lists the ingredients and quantities needed to produce it. Only products with **is_input** = **FALSE** (sellable products) would have a recipe, because raw materials themselves are not produced from other items. If a product is not manufactured (e.g. it's either an input or a finished good that is purchased complete), this can be NULL. In our design, we enforce uniqueness such that a product can have at most one recipe (one-to-one relationship), meaning a given finished product has a single standard recipe. This avoids cluttering the product table with recipe details and supports versioning by allowing inactive old recipes and linking new ones.
- **is_active:** Boolean flag indicating if the product is active in the catalog. Defaults TRUE. If a product is discontinued or not in use, set to FALSE to hide it from new transactions while preserving it historically. For example, a seasonal product might be inactive during off-season. This pattern is similar to other master tables.
- **created_at, created_by, updated_at, updated_by:** Audit fields recording when the product entry was created and last updated, and by which user. **created_at** is set on insert, **created_by** is the user who registered the product (often an admin/catalog manager). These are not null for creation, and optional for updates. They allow traceability of catalog changes (e.g. who added a new product).

****Relationships:****

- **Product–Category (N–1):** Each product belongs to one category. Many products may share the same category.

- **Product–Inventory (1–N):** A product can appear in the inventory of many branches. The `inventory` table uses `product_id` to track stock of each product per branch. So one product will have multiple inventory records (one per branch carrying it) . Conversely, each inventory record is for one product at one branch.
- **Product–Recipe (1–1 or 1–0..1):** A finished product may have at most one recipe. The optional `recipe_id` in product points to that recipe. In practice this is a one-to-one relationship: one product -> one recipe (if applicable) . We enforce uniqueness of product in the Recipe table (or handle it via a one-to-one constraint).
- **Product–RecipeItem (N–M via RecipeItem):** Products (particularly those marked as input) can appear as ingredients in many recipes, and each recipe contains many products as ingredients. This is a many-to-many relationship between Product and itself (Product as finished vs Product as ingredient) resolved through the `recipe_item` table: each `recipe_item` links one product (ingredient) to one recipe (which in turn is for one finished product) . Conceptually: a final product comprises many input products; an input product can be used in many final products’ recipes. The Recipe/RecipeItem tables handle this without an explicit direct self-relation on Product.
- **Product–SaleItem (1–N):** Each sale line (`sale_item`) references a specific product sold. A product can be sold in many different sales over time (hence many sale item records) . For example, "Coffee" product might appear hundreds of times in `sale_item` records (each a line on some receipt).
- **Product–PurchaseItem (1–N):** Similarly, each purchase order line (`purchase_item`) references a product being ordered from a supplier. An ingredient like "Flour 50kg" might appear in many `purchase_item` records across multiple orders . This tracks procurement of raw materials or even finished goods from external vendors.
- **Product–Production (1–N indirect):** If a product is a finished good that is produced in batches, the `production` table records production events of that product. A given product (e.g. "Concha Bread") can have many production lots over time . Likewise, raw material products appear indirectly in productions via inventory movements and recipe details (the product table doesn’t directly link to Produc-

tion, except possibly via the recipe; in our design the production record has a `product_final_id` referencing the finished product).

- **Product–TransferItem (1–N):** When stock is transferred between branches, each `transfer_item` specifies a product and quantity moved. A product can appear in many such transfer item records (e.g. multiple transfers) . This is analogous to `sale_item` and `purchase_item` relationships.
- **Product–InventoryMovement (1–N):** Every inventory movement record (stock ledger entry) is for a particular product, indicated by `product_id`. A product will have many movement records over time (sales, purchases, adjustments affecting it) . Filtering the movement log by product yields its full stock change history.

In summary, the Product table is referenced by many others, reflecting its central role. The DB ensures that you cannot reference a non-existent product in a sale, purchase, etc., and typically we do not delete products that have history (we mark inactive instead) to avoid breaking these links.

****Usage Scenarios:****

- **Adding a New Finished Product:** The catalog manager introduces a new sellable product, say "Glazed Donut". They choose a category (e.g. "Sweet Pastries"), set `is_input` = 0, define `measure_unit` = "pz" (pieces), set a `sale_price` (e.g. \$10.00), and because this is made in-house, they will associate a recipe either now or later . When saved, the product gets an `id_product` and is now available to use: it can be included in the POS, inventory, etc. If a standard recipe exists, they link it via `recipe_id` (or create the recipe after the product is created). From now on, every time the bakery sells or produces Glazed Donuts, those records will reference this product ID.
- **Adding a New Ingredient (Raw Material):** The purchasing manager adds a new input product, e.g. "Ground Cinnamon 1kg". They set it as `is_input` = 1, category = "Ingredients", unit = "kg", and perhaps an initial `standard_cost` per kg if known . `sale_price` would be left NULL (since it's not sold to customers) . Once saved, this product can be used in recipes (e.g. the donut recipe might include cinnamon) and can be included in purchase orders to stock it. It will also appear in inventory records for branches once stock is acquired.
- **Price Update:** If the bakery decides to raise the price of the Glazed Donut from \$10 to \$12, the manager updates the `sale_price` field for

that product. To preserve history, typically receipts store the price at time of sale in the `sale_item` record, so old sales remain at \$10. The product's current price is now \$12 for future transactions. (One could track price changes by effective dates or separate price table if needed, but in simple terms, the product table holds the current price).

- **Inventory Reorder Planning:** The `min_stock` (or branch-specific reorder points) for products help flag low stock. For instance, flour has `min_stock = 100 kg`. The system or manager can run a report of all products where current stock (summing branches or per branch) is below `min_stock` to decide purchase orders. This value can be adjusted if usage patterns change.
- **Deactivating a Product:** If a product is discontinued (say a seasonal item or a recipe changed so much it's considered a new product), the old product can be marked `is_active = 0`. It will no longer show up for new sales or production. However, past sales/productions remain linked to it. For example, "Pumpkin Pie" might be sold only in October; come November, set it inactive to hide it. Next October, you could reactivate or create a new entry if the formula changed. The integrity is preserved because we do not delete the product as long as it was used in records.

4.3 Recipe and Recipe Item (Receta y RecetaDetalle)

A **recipe** defines the formula to produce a finished product, listing ingredient quantities. It acts like a Bill of Materials for production. Each recipe is linked to one product (the final good produced) and has multiple **recipe items** that enumerate the input products and their required amounts.

		Field	Data Type	Null?	Key / Constraints	Con-	Defa
Recipe table structure:		id_recipe	BIGINT	No	PK		(auto
		product_id	BIGINT	No	FK	→	–
					product(id_product),		
					UNIQUE		
		name	VARCHAR(100)	Yes			–
		yield_qty	NUMERIC(12,3)	Yes			–
		yield_unit	VARCHAR(20)	Yes			–
		is_active	BOOLEAN	No			TRU
		created_at	TIMESTAMPTZ	No			NOW
		created_by	BIGINT	No	FK	→	–
					user(id_user)		
		updated_at	TIMESTAMPTZ	Yes			–
		updated_by	BIGINT	Yes	FK	→	–
					user(id_user)		
		Field	Data Type	Null?			
Recipe Item (ingredient line) table structure:		id_recipe_item	BIGINT	No			
		recipe_id	BIGINT	No			
		input_product_id	BIGINT	No			
		quantity	NUMERIC(12,3)	No			
		notes	VARCHAR(255)	Yes			

Field meanings (Recipe):

- **id_recipe**: Primary key of the recipe.
- **product_id**: Foreign key to the final product that this recipe produces (often called **producto_final**). This is unique in the Recipe table, enforcing a one-to-one relationship: each finished product can have at most one recipe active at a time . (If a product's recipe changes, typically the old one is marked inactive and a new recipe record is created for the same product with is_active = 1, or versioning can be handled via name or version field.) This field is mandatory and essentially links the recipe to its product.

- **name:** An optional name or label for the recipe, e.g. "Original Recipe" or "Low-sugar Variant". Often there may be only one recipe per product and this can be left null or used for versioning.
- **yield_qty:** Optional field indicating the typical output quantity this recipe produces. For example, a donut recipe might have `yield_qty = 24` if the recipe makes 24 donuts per batch . This helps scale the ingredients if producing a different amount.
- **yield_unit:** Unit for the yield quantity, corresponding to the product's measure. E.g. "pz" if yield is in pieces, or "kg" if the recipe yields a bulk quantity of dough by weight . Should be consistent with the final product's unit. Both `yield_qty` and `yield_unit` are useful for planning but are optional.
- **is_active:** Boolean flag, default TRUE. Only one recipe per product is typically active at a time. If a recipe is replaced or obsolete, set `is_active = 0` and possibly create a new recipe record for that product with `is_active = 1` . This allows maintaining old recipes for historical reference while using the new one moving forward. This field is not nullable.
- **created_at, created_by, updated_at, updated_by:** Audit fields for the recipe definition. They track when the recipe was defined and by whom (e.g. the master baker or admin entering it), and updates. These work like in other tables.

****Field meanings (Recipe Item):****

- **id_recipe_item:** Primary key of the ingredient line.
- **recipe_id:** Foreign key to the recipe this item belongs to (linking to the recipe header).
- **input_product_id:** Foreign key to a product that is used as an ingredient in the recipe. This should refer to a product with `is_input = TRUE` typically (raw materials), although conceivably a finished product could be an ingredient in another product (for example, using a sub-assembly), but in most bakery cases it's ingredients like flour, sugar, etc. The same product can appear in multiple recipe items across different recipes (e.g. "Sugar" used in many recipes) .

- **quantity:** The amount of that ingredient required by the recipe (in the units of the ingredient product). Must be ≥ 0 (hence the CHECK constraint). For example, 0.500 (kg of flour) or 5.00 (units of eggs) . These quantities are typically set such that applying the recipe yield yields the correct output. If the recipe yield is 24 pieces, these quantities produce 24 pieces.
- **notes:** Optional text notes for this ingredient. Perhaps describing special preparation (e.g. "pre-sift flour", or "chopped nuts"). Can also be used for variant instructions or substitution hints.

****Relationships:****

- **Recipe–Product (1–1):** Each recipe is associated with exactly one final product, and each final product has at most one active recipe . The one-to-one is enforced via a unique constraint on `product_id`. This separation of recipe from product allows us to not clutter the product table with multiple ingredient fields and also allows multiple versions over time . Products that require no production simply have no recipe record.
- **Recipe–RecipeItem (1–N):** A recipe has many recipe items (ingredients). Each ingredient entry links back to one recipe . When a recipe is deleted or inactive, its child items are handled accordingly (in ON DELETE CASCADE, all recipe items would delete with the recipe, but usually we won't delete recipes that are used).
- **RecipeItem–Product (N–1, ingredients):** Each recipe item references an input product (ingredient). Many recipe items (across various recipes) can refer to the same product . This is how an ingredient (like flour) connects to multiple recipes.
- **Recipe–Production (1–N):** (Optional relationship) The production records may optionally link to which recipe was used for that batch, via a `recipe_id` in the `production` table. If used, a recipe can be referenced by many production entries (e.g. every time you produce cupcakes, you note which recipe version you used) . This is useful for traceability if recipes change over time or for quality tracking. If not linking explicitly, one assumes the active recipe was used for a given production by default.
- **Product–RecipeItem (N–N via RecipeItem):** Indirect many-to-many between products: a final product has many input products

(via recipe items), and an input product can be used in many final products' recipes . RecipeItem carries the junction of these.

****Usage Scenarios:****

- **Creating a New Recipe:** When introducing a new manufactured product (e.g. "Chocolate Cupcake"), the head baker or admin creates a recipe for it. For example, they add a Recipe with `product_id = Cupcake product`, set yield = 12 cupcakes per batch (`yield_qty=12, yield_unit="pz"`), and mark it active . Then they add multiple RecipeItem lines: flour 500g, sugar 200g, cocoa 50g, etc., each linking to the respective ingredient products and specifying the quantity needed . Now, whenever production of cupcakes is planned, the system knows the required ingredients and amounts. Each recipe item is stored with the exact required quantity, allowing the system to sum up needed inputs for any given batch or to scale if the production quantity changes.
- **Viewing Ingredients (BOM):** If a user wants to know the ingredients for a product, the system can query RecipeItems by recipe. For example, viewing the "Chocolate Cupcake" entry shows all its ingredients and amounts (flour 500g, sugar 200g, etc.) by joining `recipe -i recipe_item -i product` (for ingredient names) . This is useful for understanding cost (summing standard costs of ingredients) or for preparing a batch.
- **Production Planning:** Suppose each morning the bakery decides how much to produce of each product. If the AI forecasting module suggests making 100 cupcakes today, the manager enters a production order for 100 cupcakes. The system can use the recipe's yield to scale the ingredient list: the recipe was for 12 cupcakes, so for 100, it multiplies each ingredient by 8.33 . It would thus tell the baker: for 100 cupcakes, prepare 4.17 kg flour ($500g \times 8.33$), 1.67 kg sugar, etc. Once the production is executed, the baker might adjust actual usage (e.g. a little more flour if needed). During the production record, they might log actual usage via production detail or simply note it. Regardless, the inventory system will deduct the theoretical or actual used ingredient amounts and increase finished product stock accordingly, based on the recipe ratios .
- **Recipe Versioning (Change):** If tomorrow the baker optimizes the cupcake recipe (uses less sugar, adds a new ingredient), a new recipe can be created for "Cupcake" product: mark the old recipe `is_active`

= 0 and insert a new recipe with those changes and `is_active = 1` . Future productions will reference the new recipe. Historical production records could optionally store which recipe version was used (if `recipe_id` was recorded in production), so one can trace that batches up to a certain date used Recipe A and later batches used Recipe B . If no explicit link is stored, one infers by date which recipe was active. Having multiple recipes also allows experimentation: e.g., you might test a new recipe while keeping the old one active for production. However, typically only one is active at a time per product.

- **Scaling / Costing Analysis:** Using the recipe data, the system can help calculate the cost of a product by summing the standard costs of its ingredients in the recipe. Also, it provides insight: for example, one could ask "how much flour is needed in total to produce today's plan?" by summing flour across all recipes times the production quantities. This is crucial for purchase planning (ensuring enough ingredients are on hand).

5 Inventory Module

The Inventory module manages stock counts and movements. It consists of the **inventory** table, which holds the current stock level of each product at each branch, and the **inventory_movement** table, which is a transactional log (ledger) of all stock changes (increases, decreases) with references to their causes (sales, purchases, production, transfers, adjustments). This design provides both snapshot and historical views of inventory.

5.1 Inventory (Existencias por sucursal)

The **inventory** table (sometimes called stock or existence) tracks how much of each product is available at each branch location . There is typically one row per product per branch. It may also include reserved quantities (e.g. items set aside or in orders). The structure:

Field	Data Type	Null?	KeyDefault / Con- straints
id_inventory	BIGINT	No	PK (auto)
branch_id	BIGINT	No	FK – → branch(id_branch)
product_id	BIGINT	No	FK – → product(id_product)
quantity	NUMERIC(14,3)	No	CHECK(quantity ≥ 0)
reserved	NUMERIC(14,3)	No	CHECK(reserved ≥ 0)
reorder_point	NUMERIC(14,3)	Yes	NULL
last_updated	TIMESTAMPZ	No	NOW()
UNIQUE(branch_id, product_id) (Composite unique key)			

****Field meanings:****

- **id_inventory**: Primary key (surrogate) for the inventory record.
- **branch_id**: Foreign key to the branch. Indicates which branch's stock this record represents . Not null.
- **product_id**: Foreign key to the product. Indicates which product's stock is being tracked . Not null.
- **quantity**: The current quantity on hand of the product at the branch, in the product's unit of measure. This is a running total that gets updated whenever stock is received, sold, adjusted, etc. It cannot be negative (the CHECK enforces **quantity** ≥ 0 unless negative stock is allowed, but typically we disallow negatives to indicate if stock would go below zero, the system should block the transaction or handle back-orders) . The default is 0, meaning if no stock has been added yet, quantity is zero.
- **reserved**: Quantity reserved (allocated) for pending operations. This might include items put on hold for an order, or in the context of

production, maybe ingredients allocated but not yet consumed, etc. Not all systems use a reserved field, but it's included to reflect that some stock might be committed and not free for sale. Reserved is never negative and defaults to 0. If the system handles reservations (like a customer order that will be picked up later, reserving inventory), this gets incremented for reserved items and decremented when fulfilled or released.

- **reorder_point**: An optional field that can override the product's global **min_stock** for this branch. It's a branch-specific minimum desired stock level for this product. If set, the branch should reorder or transfer in product when **quantity** falls below this level. This field is null if we either don't set a branch-specific threshold or rely on the product's general **min_stock**. It's useful if, say, one branch sells much more of a product than another, so their reordering threshold is higher.
- **last_updated**: Timestamp of the last update to this inventory record. This is updated whenever **quantity** (or reserved or other relevant fields) change (e.g. after a sale, we update the stock, so update the timestamp). Default is `NOW()` on insert. It provides quick info on how recent the stock figure is (though in a perpetual inventory system, it's always current as of the last transaction).
- **Unique (branch_id, product_id)**: This composite unique key ensures there is only one inventory record for a given product at a given branch. It prevents duplicate entries. This also effectively makes (branch, product) a candidate key and we could use it as primary key instead of an arbitrary id; however, using an ID is convenient and allows easier referencing from movements if needed.

****Relationships:****

- **Inventory–Branch (N–1)**: Many inventory rows (for different products) belong to one branch. That is, each branch will have a set of inventory records, one per product they carry or track.
- **Inventory–Product (N–1)**: Many inventory rows (in different branches) correspond to one product. So a product can have a stock entry in each branch (assuming that branch carries it; if not, either no record or record with 0, depending on approach—commonly we only create records when a branch starts carrying the product).

- **Inventory–InventoryMovement (1–N):** This is more of a logical relationship: every inventory movement references a product and branch, and it updates the corresponding inventory record. We do not necessarily directly link the tables by foreign key (since movements already have branch and product fields, they indirectly point to one inventory record). However, conceptually, an inventory record’s quantity is the sum of its initial quantity plus all movement quantities to date.

****Usage Scenarios:****

- **Stock Lookup and POS Check:** When a cashier attempts to sell a product, the system checks the `inventory.quantity` for that product at that branch to ensure there is sufficient stock. For example, a customer wants 10 croissants. The POS queries the inventory record for "Croissant" @ Branch X. If `quantity >= 10`, the sale can proceed and the inventory will be updated afterwards; if it was only 2, the system would alert insufficient stock. This ensures the store doesn’t sell more than it has (unless negative stock allowed).
- **Receiving Shipment (Purchase):** When Branch Central receives 100 kg of flour from a supplier, a purchase record is logged and an inventory movement is created (type "PURCHASE"). The inventory table for Branch Central, product "Flour", is updated: `quantity` increases by 100. The `last_updated` timestamp is set to now. If before reception quantity was 20, now it becomes 120. The composite unique key ensures we update that one row, not create duplicates. If it was the first time this branch stocked flour, we likely had an entry with 0 that now becomes 100 (or we create an entry if it didn’t exist initially).
- **Stock Deduction on Sale:** A sale of 3 units of product X at Branch Y triggers a reduction of `quantity` by 3 in Branch Y’s inventory for product X. This happens via an inventory movement of type "SALE". For example, Branch Downtown had 50 muffins, a customer buys 3, the `quantity` goes down to 47. The `last_updated` is set to now. If such a sale would cause quantity to go negative, the system should block it (enforced by the check constraint and application logic). Typically, an attempted sale that exceeds available stock results in an error unless special override (like backorder).
- **Periodic Stock Audit and Adjustment:** At day’s end or periodically, staff might count actual stock and compare with the system’s

quantity. If they find a discrepancy (say system says 10 but only 8 are found, perhaps due to spoilage or theft), they will record an **inventory_movement** adjustment (type "ADJUSTMENT" or "WASTE") for -2 with a note. This will reduce **quantity** to match reality (8), update **last_updated**, and maintain traceability by having that movement referencing who adjusted and why ("2 units waste due to burn")

- **Reserved Stock Usage:** If the bakery allows customers to order in advance or if production reservations are used, the **reserved** field comes into play. For example, a large order for 20 croissants tomorrow might reserve 20 out of today's stock. When that reservation is made, **reserved** increases by 20 (from 0 to 20), while **quantity** might remain the same physically. For fulfillment, one could consider available stock = quantity - reserved. As sales happen, the system ensures not to dip into reserved stock. Once the reserved order is fulfilled or canceled, the reserved count is decremented accordingly. The reserved field thus prevents overselling stock needed for special orders. In many systems, reserved might be managed differently, but we include it for completeness.
- **Low Stock Alert:** The **reorder_point** field can trigger alerts. Suppose for "Coffee Beans" at Branch East, **reorder_point** = 5 (kg). If after sales the quantity goes down to 4, the system can flag that as below threshold, prompting the manager to reorder beans. If not set per branch, the product's global **min_stock** could be used similarly. This helps ensure inventory is replenished in time.

5.2 Inventory Movement (MovimientoInventario)

The **inventory_movement** table is an audit log of every stock change event in the system. It serves as the ledger of inventory transactions (akin to accounting journal entries for stock). Each movement record typically corresponds to an operational event like a sale, a purchase receipt, a production usage or output, a transfer dispatch or receipt, or manual adjustments. By querying this table, one can reconstruct inventory history and investigate discrepancies.

****Structure:**** (In PostgreSQL, we might implement the **movement_type** as an ENUM for fixed categories of movement.)

Field	Data Type	Null?	Key / Con- straints	Default
id_movement	BIGINT	No	PK	(auto)
branch_id	BIGINT	No	FK →	branch(id.branch)
product_id	BIGINT	No	FK →	product(id.product)
timestamp	TIMESTAMPTZ	No		NOW()
movement_type	VARCHAR(20)	No		–
quantity_change	NUMERIC(14,3)	No		–
ref_type	VARCHAR(20)	Yes		NULL
ref_id	BIGINT	Yes		NULL
user_id	BIGINT	No	FK →	user(id.user)
note	VARCHAR(255)	Yes		NULL

(We use generic names above: some designs might name these fields differently, but concept remains.)

****Field meanings:****

- **id_movement**: Primary key of the movement record.
- **branch_id**: Foreign key to branch where this movement occurred . For example, if it's a sale at branch North, branch_id = North. For a transfer, the origin branch will log a movement with branch_id = origin, and the destination will log one with branch_id = destination.
- **product_id**: Foreign key to the product affected . E.g. product "Flour".
- **timestamp**: Date and time when the movement took place or was recorded. Not null (we need a time for ordering movements chronologically) . This could be set to the transaction time or system entry time. It's indexed for time-series queries (looking up movements in a date range).
- **movement_type**: A short code or description of the type of movement. Typical values include:
 - "SALE" – stock leaving due to a sale
 - "PURCHASE" – stock coming in from a supplier delivery

- "PRODUCTION_CONSUME" – raw material used in production (stock out)
- "PRODUCTION_OUTPUT" – finished goods produced (stock in)
- "TRANSFER_OUT" – stock sent out to another branch
- "TRANSFER_IN" – stock received from another branch
- "ADJUST_POSITIVE" or "ADJUST_NEGATIVE" – manual adjustments up or down (for corrections, waste, inventory counts)

This field allows filtering movements by nature of event and is not null . Controlled vocabulary is recommended (we could enforce via an ENUM or separate type table).

- **quantity_change**: The quantity change associated with the movement. This can be positive (stock added) or negative (stock removed) . E.g. a sale of 3 means quantity_change = -3 for that product, a purchase of 50 gives +50, a breakage of 1 is -1. We keep sign in the movement rather than separate in/out fields, as it simplifies ledger math. A CHECK could ensure it's not zero. The movement_type often implies the sign (sales are negative, receipts positive), but we record it explicitly.
- **ref_type**: Optional text field indicating the source entity or process that caused this movement . For example, "Sale", "Purchase", "Production", "Transfer", "Manual". It complements movement_type by naming the table or process. Often it's redundant with movement_type but can add clarity (movement_type might categorize by effect, while ref_type names the originating module).
- **ref_id**: Optional reference ID tying this movement to a specific record in another table (the primary key of the originating document) . E.g. for movement of type "SALE", ref_type = "Sale" and ref_id = the sale transaction ID. For a production consume movement, ref_type = "Production" and ref_id = that production batch ID. This creates an audit trail: you can navigate from the movement to the actual sale, purchase order, production batch, transfer record, etc., that caused it . These fields are nullable because not all movements have a single originating record (e.g. an adjustment might not have an external reference, or a very simple system might not use IDs).
- **user_id**: Foreign key to the user who performed or recorded this movement . This is required; every stock change should be traceable to a

staff user (or system user for automated events). For sales, it's the cashier; for purchases, maybe the storekeeper who logged receipt; for production, the baker or system (if automated); for adjustments, whoever did the count. This provides accountability and traceability.

- **note:** An optional text note for additional details . For example: "Adjustment: 2 units discarded as spoiled", or "Transfer via van on 2025-08-01", or a supplier lot number etc. It's free-form and can be NULL.

****Relationships:****

- **Movement–Branch (N–1):** Many movements happen in one branch. All movements are recorded per branch .
- **Movement–Product (N–1):** Many movements affect one product over time . Filtering by product yields its movement history (what happened to "Flour" over time).
- **Movement–User (N–1):** Many movements can be recorded by the same user (FK user_id) . E.g. one manager might do all adjustments at month-end.
- **Movement–(Source Document) (N–1 implicit):** Through the `ref_type` and `ref_id`, each movement can link to at most one source record (like a sale ID or transfer ID). This is not a formal FK because it points to various tables depending on type (hence the design uses a generic reference). But logically, e.g. all movements with `ref_type` "Sale" and `ref_id` = X are the movements generated by sale #X. We typically have exactly one movement per sale item, but sometimes a summary movement per sale if not tracking line-level. For transfers, we'd have two movements referencing one transfer ID (one negative at origin, one positive at destination). For production, multiple movements (inputs and outputs) might reference one production ID .

The `inventory_movement` table is often very large (it logs every stock change). It should be indexed by product, by branch, and by date for performance . We might partition it by date if extremely large, as recommended in the indexing strategy.

****Usage Scenarios:****

- **Sale Transaction Logging:** When a sale is completed at Branch A for 2 units of Product B, the system creates an `inventory_movement`

record: `branch_id = A`, `product_id = B`, `timestamp = now`, `movement_type = "SALE"`, `quantity_change = -2`, `ref_type = "Sale"`, `ref_id = (that sale's ID)`, `user_id = cashier's ID`. This record (along with a similar one for each item sold) serves as the official deduction log from inventory . The inventory table's quantity is decremented by 2 for product B at branch A, and that new quantity can be derived by summing all movements or by having updated it. If later someone questions why stock of B is low, one can find all "SALE" movements for product B to see how it was sold.

- **Purchase Receipt Logging:** When branch A receives a supply of 100 of product C (from a Purchase Order #123), an inventory movement is recorded: `branch_id = A`, `product_id = C`, `timestamp = now`, `movement_type = "PURCHASE"`, `quantity_change = +100`, `ref_type = "Purchase"`, `ref_id = 123`, `user_id = storekeeper's ID`, `note = maybe the supplier or invoice number`. The inventory for product C at A is incremented accordingly. Now the stock increase is traceable to PO 123 . Later, one can sum all "PURCHASE" movements to see total received from suppliers in a period.
- **Production Consumption and Output:** Suppose branch Central produces a batch of donuts. The system will log movements for each ingredient used (flour, sugar out) and the finished product in. For instance, 5 kg flour used: `branch = Central`, `product = Flour`, `type = "PRODUCTION_CONSUME"`, `quantity_change = -5`, `ref_type = "Production"`, `ref_id = 77` (production lot ID), `user_id = baker's ID` . Similarly, sugar used: another -X movement. Then finished donuts produced: `branch = Central`, `product = Donut`, `type = "PRODUCTION_OUTPUT"`, `quantity_change = +100` (pieces), `ref_id = 77`, `user = baker`. The net effect on inventory is that ingredient quantities drop and donut stock increases by 100. All are tied to production #77 for traceability . If later quality issues arise, one can see exactly what ingredients (and how much) went into lot 77 and who was responsible, via these records.
- **Stock Transfer Between Branches:** Branch X sends 5 units of product D to Branch Y. On dispatch, Branch X's inventory is reduced and Branch Y's is increased upon receipt. The system will log two movements: - At origin (X): `movement_type = "TRANSFER_OUT"`, `quantity_change = -5`, `ref_type = "Transfer"`, `ref_id = 555` (transfer ID), `user = person at X who sent`, `note maybe "sent via courier"`

. - At destination (Y): when received, `movement_type = "TRANSFER_IN"`, `quantity_change = +5`, `ref_id = 555`, `user = receiver at Y`. Both reference the same transfer ID 555. Now, a global inventory auditor can match that every `TRANSFER_OUT` has a corresponding `TRANSFER_IN`. If something goes missing in transit, one might see mismatch (e.g., -5 out but only +4.5 in, which might indicate loss or damage). The notes might indicate such a discrepancy, or there could be an adjusted final receipt entry to account for it .

- **Manual Adjustment (Waste/Count):** During stock take, 1 unit of product E is found spoiled. The manager enters an adjustment: `movement_type = "ADJUST_NEGATIVE"`, `quantity_change = -1`, `ref_type = NULL` (or "Adjustment"), `ref_id = NULL`, `user = manager`, `note = "Spoiled, discarded"` . Inventory for E is reduced by 1. This record provides justification for the loss. If an overage is found (maybe inventory count found 2 extra due to error), they might do an "ADJUST_POSITIVE" movement to align the records.
- **Auditing and Forensics:** The movement log allows auditors to answer questions like "Why is inventory of product F low?" by listing all movements for product F at that branch: e.g. 5 sold, 2 used in production, etc., and verifying if each has a valid source . If there's an unexplained drop with no movement, that indicates a missing transaction (potential issue). It also helps ensure all stock changes are accounted for – e.g. total company-wide inventory should obey: starting stock + sum of purchases + production outputs = sum of sales + production consumes + adjustments + current stock (transfers cancel out between branches globally) . This table is central to such analyses.

6 Sales Module (POS)

The Sales module handles customer sales transactions, payment records, customer info for sales, and cash register sessions. It includes the **customer** table for storing frequent customers or corporate clients, **cash_session** for tracking register openings and closings, the **sale** table for sales receipts (header), and related tables **sale_item** (line items of products sold) and **sale_payment** (payment breakdown for each sale). These tables collectively represent the POS (Point-of-Sale) functionality.

6.1 Customer (Cliente)

The **customer** table holds information about clients when needed (for example, corporate customers, loyalty program members, or anyone requiring an invoice). Most bakery sales might be walk-in consumers without records, but this table is used for those cases where you want to attach a name or account to a sale. It typically includes personal or business details.

Field	Data Type	Null?	Key / Con-straints	Default
id_customer	BIGINT	No	PK	(auto)
name	VARCHAR(100)	No		–
customer_type	VARCHAR(20)	No		–
tax_id	VARCHAR(50)	Yes		NULL
phone	VARCHAR(20)	Yes		NULL
email	VARCHAR(100)	Yes		NULL
address	VARCHAR(255)	Yes		NULL
city	VARCHAR(50)	Yes		NULL
state	VARCHAR(50)	Yes		NULL
country	VARCHAR(50)	Yes		NULL
is_active	BOOLEAN	No		TRUE
registration_date	DATETIME	No		NOW()

Field meanings:

- **id_customer**: Primary key of the customer record.
- **name**: The customer's full name (if individual) or company name (if business) . This is required. For individuals, typically "First Last"; for companies, the legal name.
- **customer_type**: A classification of customer, e.g. "PERSON" or "COMPANY", or categories like "RETAIL", "WHOLESALE" if needed . Not null. It helps determine what data is needed (companies might require tax ID, etc.) and how to address them. Could be limited to a few values.
- **tax_id**: Tax identification number (like VAT, RFC in Mexico, NIT, etc.) for invoicing . Optional, but if the customer is a company or requests an invoice, this should be filled. For an individual, could be personal ID if needed. The format might be country-specific.
- **phone**: Contact phone number, optional .

- **email:** Contact email, optional . Useful for sending receipts, promotions, or as a login for loyalty program.
- **address, city, state, country:** Physical address details, optional . These are mostly needed for issuing formal invoices or deliveries. If multiple addresses per customer were needed, a separate address table could be used; here we assume one primary address stored.
- **is_active:** Boolean indicating if this customer is active (can be selected for new sales) or not. Defaults to TRUE. If a customer should no longer be used (e.g. requested deletion for privacy, or a business closed), set to FALSE to hide them in future selection while preserving past associations .
- **registration_date:** Timestamp of when this customer record was created (signed up). Not null (defaults to current time). Useful for loyalty program enrollment date, etc. .

****Relationships:****

- **Customer–Sale (1–N):** A customer can have many sales associated with them (via `sale.customer_id`). If a sale is made to a registered customer, that sale row points here . E.g. "La Esquina Restaurant" might have dozens of sales (orders) over time. Customers who are walk-ins typically result in `sale.customer_id = NULL` or a generic "Public" customer record.
- (Other potential relations not in scope: e.g. customer loyalty points, accounts receivable, etc., but those would involve other tables.)

****Usage Scenarios:****

- **Walk-in Sale (No Customer):** A one-time customer comes and buys without providing info. In the sale record, `customer_id` is left NULL or could be set to a default "General Public" customer entry. The customer table doesn't get involved here . This is the typical case for small purchases where no invoice or tracking is needed.
- **Sale to Known Customer:** A frequent customer identifies themselves (e.g. phone number or loyalty ID). The cashier finds their customer record (say ID 42 for "Restaurante La Esquina") and attaches it to the sale . Now the sale is linked to that customer. This allows

special pricing or discounts (maybe the system applies a discount because La Esquina is a bulk buyer), and later you can filter sales by that customer to see total purchases . The system might also print their tax info on an invoice, etc.

- **New Customer Registration:** A new wholesale client wants to start buying regularly. The manager opens the customer module and creates a new record: e.g. name "Panadería Perez", type "COMPANY", enters their tax ID, address, etc. . Now on subsequent sales, this record can be selected so their invoices are properly addressed. The `registration_date` is stored to know since when they've been a client.
- **Loyalty Program:** If the bakery has a loyalty program, each member is a customer entry. For example, someone signs up with email and phone. Their purchases are recorded with their `customer_id`, which allows accumulation of points (the points themselves might be tracked in a separate table, but we know which sales belong to them). We could calculate total spending of that customer via sum of their sales. Also, having email allows us to send them promotions .
- **Inactivating a Customer:** A customer asks to be removed from mailing list or stops doing business. We can mark them `is_active = 0` . They will no longer appear in lookup for new sales or communications. However, their past sales remain in the system for records (some systems might anonymize the name if required by GDPR etc., but keep the linkage for internal sums under an anonymous ID). Generally, we prefer to inactivate rather than delete because sales link to these records .
- **Bulk Sales Analysis:** Using the link, managers can query, e.g., "How much did Restaurant La Esquina buy this month?" by summing all sale totals where `customer_id = that restaurant` . Or "Who are the top 5 customers by revenue?" by grouping sales by customer. This helps in B2B or loyalty contexts. Similarly, if doing accounts receivable, one could track how much is owed by each customer (though that would need a payments link).

6.2 Cash Session (Apertura/Cierre de Caja)

The `cash_session` table logs each opening and closing of a cash register (till) at a branch. Typically at the start of a shift or day, a cashier opens a

session with a starting cash amount; at end, they close it, counting cash and finalizing. This table ensures accountability for cash handling and is linked to sales (sales made during a session can be associated with it). Key fields include timestamps and amounts to check if expected vs actual cash match.

Field	Data Type	Null?	Key / Constraints	Default
<code>id_cash_session</code>	BIGINT	No	PK	(auto)
<code>branch_id</code>	BIGINT	No	FK → <code>branch(id_branch)</code>	–
<code>user_id</code>	BIGINT	No	FK → <code>user(id_user)</code>	–
<code>open_time</code>	DATETIME	No		NOW()
<code>close_time</code>	DATETIME	Yes		NULL
<code>opening_amount</code>	DECIMAL(10,2)	No		0.00
<code>closing_amount</code>	DECIMAL(10,2)	Yes		NULL
<code>expected_cash</code>	DECIMAL(10,2)	Yes		NULL
<code>status</code>	VARCHAR(20)	No		"OPEN"

Field meanings:

- `id_cash_session`: Primary key.
- `branch_id`: The branch where this register session took place. Not null (each session is for a particular store's till).
- `user_id`: The user (cashier) who opened (and usually who will close) this session . Typically one cashier per session.
- `open_time`: Timestamp when the session was opened (the moment the cashier started the register) . Set when session is created.
- `close_time`: Timestamp when the session was closed. Null until the session is actually closed (then set) . If a session is currently ongoing, `close_time` is NULL and `status` might be "OPEN".
- `opening_amount`: The initial cash in drawer at open, entered by the cashier. E.g. they start the day with \$100.00 float . Not null (even if zero).
- `closing_amount`: The actual cash counted in the drawer at closing time, entered by the cashier or supervisor. This is optional until they close (NULL while session is open) . At close, they input how much cash is physically present.

- **expected_cash:** The system-calculated expected cash at close, based on transactions during the session. The system can sum all cash payments (from sale_payment) made during that session and add opening_amount to predict how much should be in drawer . We store it at close for record. This is optional (could be computed on the fly, but storing simplifies comparisons).
- **status:** Status of session, e.g. "OPEN", "CLOSED", maybe "PENDING" etc. Defaults "OPEN" when created. Once closed, set to "CLOSED". This helps ensure only one open session per register/user combination at a time.

****Relationships:****

- **CashSession–Branch (N–1):** Many sessions occur in one branch (each register open each day) .
- **CashSession–User (N–1):** Many sessions can be opened by the same user over time (each shift they work) . Each session has one opener (and usually that same user closes it, though if someone else closes due to shift change, this design doesn't capture two users; one might extend model for multiple users handing off, but here we assume one user).
- **CashSession–Sale (1–N, indirect):** We might not have an explicit foreign key in sale to session in our model (depending on design). However, it's very useful to know which sales happened in which session (so we can sum their cash amounts). Many systems do include `sale.session_id`. Let's assume we do have that: each sale record has `session_id` referencing `cash_session`. Then a session (open to close) has many sales linked to it . This helps to group the sales to compute expected cash. If not stored, one could derive by sale timestamps between open and close time per branch, but explicit linkage is cleaner. We will consider that likely sale table includes `session_id` (though our earlier sale structure outline hasn't been done yet; we will include it).

****Usage Scenarios:****

- **Opening the Day's Register:** At 8:00 AM, cashier Juan arrives at the downtown branch, logs in and selects "Open Cash Register". He enters the opening amount, say \$100. A new `cash_session` row is created: `branch = Downtown`, `user = Juan`, `open_time = 8:00`, `opening_amount = 100.00`, `close_time NULL`, `status "OPEN"` . Juan can

now start making sales. The system might record the session ID in each new sale.

- **Sales during the Session:** Throughout the morning, Juan processes several sales (cash and card). Each sale record is stamped with `cash_session_id = (Juan's session)` and `cashier_id = Juan`. For example, sale #200 is cash \$50, sale #201 is card \$30, sale #202 cash \$20, etc. The system accumulates that during this session, total cash payments = \$70, card = \$30. (We either compute on the fly or each sale payment references session via `sale-¿session-¿id` relationship.)
- **Closing the Register:** At 2:00 PM, Juan's shift ends. He hits "Close Cash Register". The system calculates expected cash in drawer = opening \$100 + cash sales \$70 - any cash payouts (if recorded) = \$170. Juan counts the drawer physically. Suppose he finds \$169.50. He enters that as `closing_amount`. The session record is updated: `close_time = 14:00`, `closing_amount = 169.50`, `expected_cash = 170.00`, `status = "CLOSED"`. The system may flag a shortage of \$0.50. This difference might be written to note or shown to a supervisor .
- **Cash Reconciliation and Audit:** Later, a manager can review session records. For Juan's session, they see expected 170 vs actual 169.50 (perhaps noted in a report or they manually note the \$0.50 short in note field if we had one). They can then ask Juan or account it as a minor cash short. If large discrepancies occur regularly, this table provides accountability per cashier. Also, sessions tie to sale records, so one can list all sales in a session to find if any large cash transaction was potentially mishandled.
- **Multiple Sessions in a Day or Multiple Registers:** If the bakery has multiple cash registers or shifts, each will have its own session record. e.g. an afternoon cashier opens a new session with her own float when Juan leaves. Each register can either be a separate user login or share user but presumably separate sessions. We might differentiate registers by user or could have a register ID in session if needed (not in our fields, assuming one user one register at a time).
- **Preventing Forgotten Closure:** By having this table, at end of day we can check that all sessions that opened have been closed. If a session is still open past a store's closing time, an alert can remind to close it. Also, by linking sales to sessions, we ensure no sales happen

outside a session. Some systems might even require an open session before allowing cash sales, etc.

6.3 Sale and Sale Item (Venta y Detalle)

The **sale** table represents a sales transaction (like a receipt or invoice header). Each sale typically has one or more items (line entries of products sold) recorded in **sale_item**, and possibly references to a customer and the cashier. The sale header contains summary info (total amounts, timestamps, etc.).

	Field	Data Type	Null?	Key / Default	
				Con-	straints
Sale table structure:	id_sale	BIGINT	No	PK	(auto)
	branch_id	BIGINT	No	FK	→ branch(id_branch)
	cashier_id	BIGINT	No	FK	→ user(id_user)
	session_id	BIGINT	Yes	FK	→ NULL cash_session(id_cash_session)
	customer_id	BIGINT	Yes	FK	→ NULL customer(id_customer)
	datetime	DATETIME	No		NOW()
	status	VARCHAR(20)	No		'COMPLETED'
	total_amount	DECIMAL(10,2)	No		0.00
	notes	VARCHAR(255)	Yes		NULL
Sale_Item table (line items of products in a sale):	Field	Data Type		Null?	
	id_sale_item	BIGINT		No	
	sale_id	BIGINT		No	
	product_id	BIGINT		No	
	quantity	DECIMAL(10,3)		No	
	unit_price	DECIMAL(10,2)		No	
	total_line	DECIMAL(10,2)		No	

(For simplicity, fields like tax, discount per line are omitted here but could be added if needed. Also, currency is assumed consistent.)

****Field meanings (Sale):****

- **id_sale:** Primary key (receipt number).
- **branch_id:** The branch where the sale took place (so we can separate sales by location) .
- **cashier_id:** User ID of the cashier who processed the sale . This links to the person responsible.
- **session_id:** (Optional) Cash session during which this sale occurred. If recorded, it references the active **cash_session** at time of sale. Helps group sales per session for cash reconciliation . Null if not applicable (e.g. an online order or if outside normal session).
- **customer_id:** (Optional) Customer associated with the sale, if any. Null for walk-in or anonymous sales . If a corporate client purchase, this links to their customer record.
- **datetime:** Date and time of sale. Useful for chronological order, reports by day, etc. Not null (set when transaction is completed).
- **status:** Status of sale. Normally "COMPLETED" for finalized sales. Could also have "CANCELLED", "REFUNDED", "ON HOLD" etc. In our design, default "COMPLETED". If a sale is voided or cancelled, we might mark it here (and possibly handle stock return).
- **total_amount:** Total monetary amount of the sale (sum of all items minus any discounts, etc.). We store this redundantly for quick reporting . It can be calculated from sale items but storing is convenient for receipts. This is the gross total (if taxes included, etc., depending on region).
- **notes:** Optional notes about the sale (e.g. "Special discount applied", or "Customer requested later pickup", etc.). Could also store invoice reference or external transaction ID if needed.

****Field meanings (Sale_Item):****

- **id_sale_item:** PK for the line item.
- **sale_id:** Foreign key to the sale (receipt) this line belongs to. Not null; on sale deletion, cascade remove items (since items have no meaning without a sale) .

- **product_id:** Foreign key to product sold in this line . Not null (the product must exist).
- **quantity:** Quantity of the product sold. Default 1. Could be fractional (depending on product; e.g. 0.5 kg of something) so DECIMAL with some scale for weight is used, or integer if only discrete pieces. For bread pieces it's integer, but for things sold by weight like cake slices by weight, decimal is needed.
- **unit_price:** The price per unit at the time of sale. This is copied from the product's current price, but stored to capture that moment's price . If the product price changes later, past sale records remain accurate with stored unit_price. Also reflects any line-specific discount (if a discount per line was given, it might adjust unit price or we could have a separate discount field).
- **total_line:** The total price for that line (quantity * unit_price, minus any per-line discount). We store it for convenience, which also allows detection of rounding issues. Should equal quantity * unit_price in absence of other factors. Storing helps quickly summing receipts without recalculating each line.

****Relationships:****

- **Sale–Branch (N–1):** Many sales occur at one branch (sales table has branch_id) .
- **Sale–User (Cashier) (N–1):** Many sales processed by one cashier (cashier_id) .
- **Sale–CashSession (N–1):** Many sales can link to one cash session (session_id), if tracked .
- **Sale–Customer (N–1, optional):** Many sales can be associated with the same customer (customer_id) .
- **Sale–SaleItem (1–N):** One sale has multiple sale_item records (one per product line) . On delete of sale, items are removed (ON CASCADE).
- **SaleItem–Product (N–1):** Many sale items (across sales) refer to one product . This tracks product popularity etc.

- **SaleItem–Sale (N–1):** Each sale item belongs to one sale (the header)
- **Sale–SalePayment (1–N):** We haven’t described sale_payment table yet, but conceptually a sale can have multiple payments (cash, card, etc.). There will be a SalePayment table linking to sale. (We’ll do next.)

****Usage Scenarios:****

- **Basic Sale Transaction:** A customer buys 2 croissants and 1 coffee at Branch A, paying cash. The system creates a sale record: branch=A, cashier=John, session=John’s open session, customer=NULL (walk-in), datetime now, status "COMPLETED". It then creates two sale_item records: one for croissant (product=Croissant, quantity=2, unit_price= \$1.00, total_line=\$2.00) and one for coffee (product=Coffee, quantity=1, unit_price=\$3.00, total_line=\$3.00). The sale.total_amount is stored as \$5.00 (2+3) . Then a sale_payment (to be described) is recorded for \$5.00 cash. Inventory movements for -2 croissants and -1 coffee are triggered. Now the receipt can be printed with these details.
- **Sale with Invoice:** A company orders baked goods totalling \$100. The sale is recorded with customer_id referencing that company’s record, so an invoice can be issued in their name . Perhaps payment is on account (store in sale_payment as credit). Later, one can filter sales by that customer to see their purchases.
- **Cancelled Sale:** A cashier mistakenly rang up a sale or a customer returned items immediately. To void it, the sale.status might be set to "CANCELLED". We may either delete the sale and its items or keep it with status cancelled for audit. A typical practice: mark cancelled, set all quantities to 0 or do not generate inventory movements. Or if it was already recorded, we might record reverse movements. Our model can accommodate by using status and perhaps recording a refund payment with negative amount or separate table. In any case, sale table’s status can differentiate.
- **Sales Reporting:** This table enables various reports: total sales per day (sum of total.amount for that day per branch), top-selling products (by joining sale_item), average ticket size, etc. . Because sale_item has line detail, one can sum quantities by product across sales to see, e.g., 500 croissants sold today.

- **Audit and Traceability:** The link to cashier and session means if there's a cash discrepancy, one can list all sales under that session (maybe a large cash sale was voided and not properly accounted, etc.). The link to customer means any sale can be traced to who bought it if needed (for instance, recall of a product batch, you could contact customers by looking at sales of that product and checking attached customers).
- **Discounts and Specials:** While not explicitly modeled in fields above, if a discount is applied, one could either reduce `unit_price` (like a promotion making coffee \$2 instead of \$3) and thus `total_line` reflects discount, or one could add a negative line item for "Discount". Alternatively a separate discount field in sale table could store a global discount. In absence, adjusting prices directly is simplest for example. The note field could annotate "10% discount applied for loyalty".
- **Multiple Payment Methods:** If a sale is split between cash and card, the `sale_payment` table (next) will reflect two payment entries summing to `total_amount`. The sale record remains single, with `total`. This is common scenario which `sale_payment` handles.

6.4 Sale Payment (Pago de Venta)

The **sale_payment** table records payments made for a sale, allowing multiple methods per sale (e.g., part cash, part credit card). Each sale can have one or more associated payments that should sum to the sale's total amount. This table typically stores payment type, amount, and reference (e.g., last 4 digits of card or transaction ID).

Field	Data Type	Null?	Key / Constraints	Con-	Default
<code>id_sale_payment</code>	BIGINT	No	PK		(auto)
<code>sale_id</code>	BIGINT	No	FK sale(id_sale) ON DELETE CASCADE	→	—
<code>method</code>	VARCHAR(20)	No			—
<code>amount</code>	DECIMAL(10,2)	No			0.00
<code>reference</code>	VARCHAR(50)	Yes			NULL

Field meanings:

- `id_sale_payment`: Primary key.

- **sale_id:** Foreign key to the sale this payment applies to. If a sale is deleted, its payments are too (cascade).
- **method:** Payment method, e.g. "CASH", "CARD", "VOUCHER", "ONLINE" etc. . Not null. Controlled vocabulary recommended.
- **amount:** The amount paid with this method . Must be positive for payments (for refunds or change given might be negative, but typically we handle change differently and refunds as separate sale with negative items or a negative payment entry if recording cash returned).
- **reference:** Optional reference info. For card, could store the authorization code or last 4 digits, or transaction ID from a payment gateway; for vouchers maybe voucher code; for an online payment maybe transaction ID. For cash, could be empty or possibly note like "change given X" if we use it for such.

****Relationships:****

- **SalePayment–Sale (N–1):** Many payment entries can belong to one sale. Typically 1 or 2 for splits .

****Usage Scenarios:****

- **Single Payment (Cash):** A \$50 sale paid fully in cash. The system creates a sale_payment: sale_id = that sale, method="CASH", amount=50.00, reference NULL. This ties the sale to the cash session for reconciliation. At session close, system sums all sale_payment of method "CASH" in that session to compute expected cash in drawer .
- **Split Payment:** A \$100 sale where customer pays \$70 by card and \$30 by cash. Two sale_payment records: (sale_id, method="CARD", amount=70.00, reference maybe "Auth12345"), and (sale_id, method="CASH", amount=30.00) . The sale.total.amount is still 100.00. At session end, the cash part counts towards expected cash, and card part can be reconciled with card settlement reports by summing all card payments (which should match what the payment processor says).
- **Change Given Handling:** If a customer gives \$20 for a \$18 purchase, some systems might log sale_payment cash=18 and note change given 2 (the change isn't a payment from customer, it's reverse). Usually, we wouldn't record change as a separate negative payment; expected cash logic would account the cashier should have received \$20 and given \$2,

so net \$18 increase, which is exactly the sale amount. So we just record the \$18 as cash revenue. The extra \$2 received and returned doesn't affect net and doesn't need to be recorded in sales (some POS systems track it for drawer transactions, but that might be outside sale table).

- **Refund or Cancellation:** If a sale is cancelled or refunded, we might either remove payments or add negative payments. Typically, for a refund scenario, one could create a new sale with negative amounts or mark existing sale as refunded. If marking existing, one could add a negative payment line indicating money returned. For example, method "CASH", amount = -50 if refunding \$50 in cash, reference "Refund on 2025-08-16". This would reduce that day's cash total. However, design might prefer separate refund transactions. We'll not complicate further; approach depends on business rules.
- **Payment Reporting:** This table allows to report totals by payment method for a period (how much was paid in cash vs card etc.). Summing sale_payment by method and date can yield, e.g., "Cash sales today \$X, Card sales \$Y", which is useful for deposit planning and verifying with bank card deposit totals.
- **Integration with Accounting/AR:** If some sales are on credit (like the customer will pay later), we could represent that by a payment method "ACCOUNT" or "INVOICE" for the amount, meaning no actual money collected now. Later, when they pay, it might be recorded as a separate receipt not directly in this table (or as adjusting entries). But at least the sale record will show method=INVOICE amount=full sale, signifying accounts receivable was created.

This covers the sales module: we've described how administrative details like sessions and customers tie in, and how each sale and its items and payments are recorded for a complete audit trail of what was sold, to whom, by whom, when, and how it was paid .

7 Purchases Module

The Purchases module deals with acquiring supplies from vendors. It includes the **supplier** table (for vendor information), and the **purchase** and **purchase_item** tables to record purchase orders or receipts of inventory from those suppliers. In essence, this is the mirror of sales but on the input side.

7.1 Supplier (Proveedor)

The **supplier** table lists providers of ingredients or goods to the bakery. It stores contact info and audit fields similar to customers.

Field	Data Type	Null?	Key / Constraints	Default
<code>id_supplier</code>	BIGINT	No	PK	(auto)
<code>name</code>	VARCHAR(120)	No		–
<code>contact_name</code>	VARCHAR(100)	Yes		NULL
<code>phone</code>	VARCHAR(20)	Yes		NULL
<code>email</code>	VARCHAR(100)	Yes		NULL
<code>address</code>	VARCHAR(255)	Yes		NULL
<code>city</code>	VARCHAR(50)	Yes		NULL
<code>state</code>	VARCHAR(50)	Yes		NULL
<code>country</code>	VARCHAR(50)	Yes		'MX'
<code>tax_id</code>	VARCHAR(50)	Yes		NULL
<code>is_active</code>	BOOLEAN	No		TRUE
<code>created_at</code>	DATETIME	No		NOW()

(We include a `contact_name` field for a representative or salesperson of the supplier, which Spanish doc had as 'contacto').

Field meanings:

- `id_supplier`: PK.
- `name`: Supplier name (e.g. "Harinas del Norte S.A.") . Required and typically unique.
- `contact_name`: A contact person's name at the supplier (optional) .
- `phone`, `email`: Contact info (optional) . Could be for the sales rep or general office.
- `address`, `city`, `state`, `country`: Supplier address details (optional). Useful for mailing POs or record keeping . Country default 'MX' (assuming Mexico), can be changed if needed.
- `tax_id`: Supplier's tax identification number (if needed for invoices to them or from them) . Optional.
- `is_active`: Boolean, default TRUE. If a supplier is no longer used, mark inactive to hide from selection (like customers) .
- `created_at`: Timestamp when added.

****Relationships:****

- **Supplier–Purchase (1–N):** One supplier can have many purchase orders/records associated (purchase.supplier_id) . All purchases from a given supplier can be listed via this link.

****Usage Scenarios:****

- **Registering Suppliers:** When starting the system, the manager enters main suppliers: e.g., Flour Co, Dairy Inc, Packaging LLC with their contacts . Later, to create a purchase order, they will select from these suppliers.
- **Supplier Contact Update:** If a supplier changes rep or phone, update that record. If a supplier goes out of business or is replaced, mark them inactive. Past purchase records remain linked but new orders cannot be placed to an inactive supplier easily (they could override if needed).
- **Query Purchase History by Supplier:** We can answer "How much did we buy from Harinas del Norte this year?" by summing all purchase totals where supplier is Harinas . This helps negotiation (like "we purchased X amount from you").
- **Preferred Supplier per Product:** Not explicitly in fields, but one might note if a certain supplier is primary for a certain product. Spanish doc suggests possibly storing a preferred supplier ID in product, but we didn't include it. Alternatively, one could deduce from most purchases of that product which supplier is used. If needed, a separate ProductSupplier mapping could exist, but in our scope not necessary.
- **Deactivating Supplier:** If the bakery stops dealing with a supplier (too expensive etc.), mark them inactive . New POs will exclude them. If later needed again, reactivate.

7.2 Purchase and Purchase Item (Compra y Detalle)

The **purchase** table records orders or receipts of inventory from suppliers. It includes which branch (destination) got the goods, which supplier, dates of issue and receipt, status, totals, and an audit trail. The **purchase_item** table lists each product and quantity ordered/received in that purchase. This functions analogously to sale/sale_item but on the procurement side.

	Field	Data Type	Null?	Key / constraints	Con-	Default
Purchase structure:	id_purchase	BIGINT	No	PK		(auto)
	branch_id	BIGINT	No	FK	→	–
				branch(id_branch)		
	supplier_id	BIGINT	No	FK	→	–
				supplier(id_supplier)		
	issue_date	DATETIME	No			NOW()
	receive_date	DATETIME	Yes			NULL
	status	VARCHAR(20)	No			'ISSUED'
	total_amount	DECIMAL(10,2)	No			0.00
	notes	VARCHAR(255)	Yes			NULL
Purchase_Item structure:	created_by	BIGINT	No	FK	→	–
				user(id_user)		
	created_at	DATETIME	No			NOW()
	Field	Data Type	Null?	Key / constraints	Con-	Default
	id_purchase_item	BIGINT	No	PK		(auto)
	purchase_id	BIGINT	No	FK	→	–
				purchase(id_purchase)		
				ON DELETE CASCADE		
	product_id	BIGINT	No	FK	→	–
				product(id_product)		
	quantity_ordered	DECIMAL(12,3)	No			0
	quantity_received	DECIMAL(12,3)	Yes			0
	unit_cost	DECIMAL(10,4)	No			0
	total_line	DECIMAL(10,2)	No			0

Field meanings (Purchase):

- **id_purchase**: Primary key (purchase order number or entry ID).
- **branch_id**: The branch that will receive or use these supplies (destination of the purchase) . If the business has a central warehouse, that could be the "branch" here or a special branch record.
- **supplier_id**: The supplier from whom items are ordered . Not null.

- **issue_date:** Date/time the purchase order was issued or created . Default current time.
- **receive_date:** Date/time goods were actually received. Null until the order is delivered (if this table covers both ordering and receiving). Once the stock arrives and is checked in, we fill this date . If partial, maybe update when fully closed.
- **status:** Status of the order. Examples: "ISSUED" (or "PENDING") when created, "RECEIVED" when fully delivered, "PARTIAL" if partially delivered, "CANCELLED" if the order was voided, etc. Our default 'ISSUED' meaning order placed but not yet received .
- **total_amount:** Total cost amount of the purchase (sum of line items, maybe including taxes/shipping if any) at the time of completion. Like sales, storing total is convenient . This is updated when the order is completed/received or possibly at creation if all known costs.
- **notes:** Optional notes such as terms, special instructions, or internal comments (e.g. "Urgent, needed by Monday", or "Supplier will deliver in two batches", etc.).
- **created_by:** User who entered the purchase record (e.g. the inventory manager) .
- **created_at:** Timestamp of entry creation.

****Field meanings (Purchase_Item):****

- **id_purchase_item:** PK for line item.
- **purchase_id:** Foreign key to purchase order record. Items cascade delete with order.
- **product_id:** The product being ordered/received . Even if it's a raw material, it's listed as a product in our catalog.
- **quantity_ordered:** Quantity ordered. The expected amount supplier should deliver . Not null (even if 0 likely not valid).
- **quantity_received:** Quantity actually received. Null until delivered. If everything came as ordered, then set = quantity_ordered. If partially delivered or some short, that is recorded here (and possibly the order stays open or a backorder is created, but within this design if partial

we can note what came; if fully not delivered, maybe we adjust status). For example, ordered 100, received only 90, put 90 here, and either leave order open or adjust.

- **unit_cost:** The cost per unit for this item on this order . This might be from the supplier price list or negotiated. Storing it locks in that purchase price, as costs can fluctuate. In 4 decimal places to allow currency and small per-unit costs (like cost per gram etc.).
- **total_line:** Total cost for this line = (quantity_received or ordered * unit_cost). Possibly initially computed with ordered, then final with received. But ideally if partial, maybe we recalc when closed. This field helps sum easily to the purchase total and can reflect actual received cost.

****Relationships:****

- **Purchase–Branch (N–1):** Many purchase orders made for one branch’s supplies .
- **Purchase–Supplier (N–1):** Many purchases from one supplier .
- **Purchase–User (Created_by) (N–1):** Many purchase records entered by one user (the system may allow track who logged it) .
- **Purchase–PurchaseItem (1–N):** One purchase order has multiple line items (products) .
- **PurchaseItem–Product (N–1):** Many purchase line items can refer to the same product (over different orders) .
- **Purchase–InventoryMovement (1–N):** Not a direct foreign key, but logically when a purchase is marked received, inventory_movement entries are generated for the incoming stock referencing that purchase (ref_type "Purchase", ref_id = purchase id). This ties the financial record of receiving to the physical stock ledger .

****Usage Scenarios:****

- **Placing a Purchase Order:** The inventory manager at Central branch sees flour is low. They create a Purchase: branch=Central, supplier=Harinas del Norte, issue_date=today, status="ISSUED", and add items: e.g. 1000 kg flour at \$0.30/kg, 200 kg sugar at \$0.25/kg. The purchase_items: flour line with quantity_ordered=1000, unit_cost=0.30,

total_line=300.00; sugar line with quantity_ordered=200, cost 0.25, total_line=50.00. Purchase.total_amount = \$350.00 . created_by= manager. This order might be printed or sent to supplier.

- **Receiving Inventory:** A week later, supplier delivers only 900 kg flour and 200 kg sugar (flour short 100 kg). The manager updates the Purchase: mark receive_date=today, status maybe "RECEIVED" if closing it (or "PARTIAL" if expecting remainder later). They update purchase_item for flour: quantity_received=900 (was 1000 ordered), sugar received 200 matches ordered. Now they finalize: The system might adjust total_line for flour to $900 \times 0.30 = 270.00$, sugar stays 50.00, purchase.total_amount becomes \$320.00 for what arrived. They may note in purchase.notes "Short 100kg, supplier will deliver later." Inventory movements are created: +900 kg flour, +200 kg sugar into Central's inventory , referencing this purchase ID. If supplier will not deliver remainder, the PO can be closed with short. If they will, maybe keep status "PARTIAL" and create a backorder entry or leave open. In a simple model, one might either adjust ordered down to 900 or mark the rest cancelled.
- **Inspecting Purchase History:** Later, if someone asks "how much flour did we buy last quarter and from whom?" we can query purchase_item for product=flour over that date range, sum quantity_received and group by supplier. Or just sum purchase totals by supplier for that period for a cost summary .
- **Payments to Suppliers / Accounts Payable:** Payment to suppliers is not tracked here, but could be integrated. Likely outside the immediate operational DB (accounting handles it). But one could add fields like paid_status or link to payment records. Spanish doc hints at accounts payable table not detailed here . In absence, we use this for receiving and rely on external for financial settlement.
- **Soft Deletes / Edits:** If a purchase is cancelled before any receipt, we could delete it or mark status "CANCELLED" and not do any inventory moves. If an entry mistake was made, and we need to adjust, possibly the manager can edit before marking received. After receipt and inventory update, changes should be avoided (or would require corresponding inventory corrections).
- **Audit and Trazability:** Each incoming stock increase in inventory can be traced to a purchase and supplier via inventory_movement (ref

to purchase ID) . If an ingredient batch is problematic (like contaminated flour), the bakery can see which supplier provided it and which production lots used it (via purchase and recipe usage linking, albeit indirectly).

- **Multi-branch Ordering:** If branches order individually from suppliers, `branch_id` varies. If a central warehouse orders for all and then distributes, they'd have `branch=warehouse` on purchases, then internal transfers to branches. Our model supports either approach.

8 Production Module

The Production module logs the baking or production of goods. It includes the **production** table for each batch or lot produced, and possibly a **production_detail** table if we record actual ingredient usage per batch (beyond the standard recipe). In our design, we have a **production** table which references the final product, quantity produced, the recipe used, and users involved. We also allow a **production_detail** table to capture actual ingredient usage if needed for variance analysis.

8.1 Production (Lote de Producción)

The **production** table records each production run of a product at a branch, including what was made, how much, when, and by whom. It links to the recipe (if multiple versions exist) and triggers inventory updates for outputs and possibly consumption (which can also be logged via movements with production reference).

Field	Data Type	Null?	Key / Con- straints	Default
<code>id_production</code>	BIGINT	No	PK	(auto)
<code>branch_id</code>	BIGINT	No	FK →	— branch(id_branch)
<code>product_final_id</code>	BIGINT	No	FK → prod-	— uct(id_product)
<code>recipe_id</code>	BIGINT	Yes	FK →	NULL recipe(id_recipe)
<code>quantity_produced</code>	DECIMAL(12,3)	No		0.000
<code>unit</code>	VARCHAR(20)	No		—
<code>timestamp</code>	DATETIME	No		NOW()
<code>responsible_id</code>	BIGINT	No	FK →	— user(id_user)
<code>approved_by</code>	BIGINT	Yes	FK →	NULL user(id_user)
<code>notes</code>	VARCHAR(255)	Yes		NULL

****Field meanings:****

- **id_production:** Primary key (production batch identifier).
- **branch_id:** The branch where this production took place (the bakery location) .
- **product_final_id:** The final product that was produced (e.g. "Concha Bread") . This must be a product with `is_input = FALSE` presumably (a sellable product).
- **recipe_id:** The recipe used for this batch (if multiple recipes possible or to document which version) . Nullable if not applicable or only one recipe exists (then it's redundant). If recipe versioning is in use, this records exactly which formulation was followed.
- **quantity_produced:** Quantity of final product produced in this lot . E.g. 100 (pieces) or 50 (kg of dough).
- **unit:** Unit of measure for the produced quantity (should match the product's unit, e.g. "pz" or "kg") .
- **timestamp:** When the production was finished/recorded (e.g. out of oven time or time logged) .

- **responsible_id**: The user (baker) responsible for this production run . Not null. This gives accountability (who made the batch).
- **approved_by**: If there's a quality control step, the user who approved the batch (e.g. a supervisor) . Nullable if no explicit approval or if not used. Could also be used to mark final check of quality.
- **notes**: Optional notes, e.g. "Oven temp issues, took longer", or batch number, or if some pieces were wasted ("5 pieces burnt") .

****Relationships:****

- **Production–Branch (N–1)**: Many production batches happen in one branch (each branch's daily productions) .
- **Production–Product (N–1)**: Many production records for the same final product over time (e.g. produce conchas daily).
- **Production–Recipe (N–1)**: Many production lots might refer to one recipe (especially if the recipe doesn't change) .
- **Production–User (Responsible) (N–1)**: Many productions done by one user (baker) .
- **Production–User (Approved) (N–1)**: Many productions could be approved by one supervisor .
- **Production–InventoryMovement (1–N)**: Not direct FKs but logically: each production should trigger:
 - One (or more) positive inventory movement(s) for the output product (quantity_produced added to inventory with ref_type "Production", ref_id = production id) .
 - Multiple negative inventory movements for each ingredient consumed, with ref to this production (to deduct flour, etc.) .

This ensures traceability in inventory ledger of production usage.

- **Production–ProductionDetail (1–N)**: If we implement a production_detail table to log actual used quantities per ingredient for this batch, one production has many detail records (similar to recipe_items but actual values) .

- **Production–RecipeDetail (N–N indirect via production_detail or via recipe if using standard):** The relationship between a production and the ingredients is through either assuming the recipe or logging detail if actual deviates. Without production_detail, one might assume consumption = recipe quantities * batches, but if differences (waste etc.), that’s why production_detail table can capture actuals.

(Production_detail table fields would be: id, production_id, product_input_id, quantity_used, maybe notes for each usage variation. But our user request suggests optional, not explicitly required in final doc, but we’ve room to mention concept.)

Usage Scenarios:

- **Daily Production Entry:** Each morning, the baker at Branch Center produces 100 bolillos (rolls). After finishing, they log a Production: branch=Center, product_final=Bolillo, recipe=Bolillo standard recipe (id 5 let’s say), quantity_produced=100, unit=”pz”, timestamp=2025-08-16 05:30, responsible=Alice (baker), approved.by=Bob (manager who tasted one), notes=”Baked 100, good quality” . When saving this, the system:
 - Creates an inventory movement: +100 to inventory of Bolillo@Center with ref ”Produccion” id of this record .
 - Option 1: Automatically deduct ingredients as per recipe: e.g. -X kg flour, -Y yeast from inventory with ref to this production (assuming standard recipe consumption) .
 - Option 2: If using production_detail, the baker enters actual ingredients used: maybe flour slightly more than theoretical, etc., in production_detail and corresponding inventory movements are based on those actual values .

Now stock levels reflect new product added and ingredients reduced.

- **Waste in Production:** Suppose out of 100 planned, 2 rolls burned and were discarded. How to record? A few ways: - In production.notes, Alice writes ”2 pieces discarded due to burn”. They still might say quantity_produced=100 (since technically 100 were made, but only 98 are sellable). Or they could set quantity_produced=98 as actually added to inventory. Alternatively, record 100 produced and then make an inventory adjustment of -2 with note ”waste from batch”. - Another approach: use production_detail or an output scrap

field to note that out of 100, 98 good, 2 waste. Our model doesn't have explicit scrap field, but a practice could be: log production of 98 (the good output) to inventory, and note the 2 waste. This is left to operational convention. The Spanish doc scenario suggests either adjust final count or do separate waste adjustment .

- **Quality Approval:** If a supervisor must approve batches, they login after production, inspect, and if OK, update this record's approved_by with their user id. If a batch fails quality (e.g. burned), they might not approve and maybe that batch won't be added to inventory (or will be scrapped). Possibly state can be inferred: if approved_by is not null, it passed; if not, maybe it didn't or waiting. Spanish mention either deducing by approved_by or adding status. We have no explicit status field, but we can interpret if needed (or add if desired).
- **Production Planning and Traceability:** The production records allow analysis: "how many of each product do we produce per day" by summing quantity across records . Also, connecting to forecast: we might compare forecast vs actual production by linking pronostico to production by date/product. Also, if something like contamination occurs in an ingredient lot, using inventory movements we can find which production lots used that ingredient around that time (but that requires linking ingredient lot information, which we don't explicitly capture here aside from dates; some systems might incorporate lot tracking, which we did not).
- **Production vs Recipe Variance:** If production_detail is used to capture actual usage (some Spanish text suggests if wanting to compare theoretical vs real usage) , then: - For each ingredient, we store actual used. Later one can compare with recipe expected (e.g. recipe says 10kg flour, actual used 10.5kg, so 0.5kg extra usage maybe due to spillage). - This helps identify consistent variances which could mean recipe needs update or there's waste to address. - Our model doesn't require production_detail for base functioning, but it's a possible extension. If not implemented, one assumes exact recipe quantities were used.
- **Indirect Link to Sales:** There is no direct link in our schema between a production lot and the sales that eventually sell those items (since FIFO etc. not tracked). However, in analysis you might compare daily production vs daily sales of a product (e.g. produced 100,

sold 90, leftover 10 likely wasted or next day). The data is there to do that aggregate check: production yields put into inventory and sales take out. Over time, if systematically leftover or shortage, forecasting adjustments can be made. Spanish mentioned linking production to sales indirectly for leftover analysis .

8.2 Production Detail (Uso Real de Insumos) – optional

(Optional extension: ProductionDetail if tracking actual ingredient usage per production lot.)

If the bakery wants to record exactly how much of each ingredient was actually consumed for each production batch (to compare with the recipe or to adjust inventory precisely in case of variations), a **production_detail** table can be used. It would have fields: **id_production_detail**, **production_id** (FK to production), **product_input_id** (FK to product, the ingredient), **quantity_used**, and maybe notes. This is similar to a recipe's ingredient list but for the specific instance . When a production is recorded, the system could auto-fill production_detail with the recipe quantities as default, and the baker can adjust if they know they used a bit more or less.

If implemented, inventory deduction can come from these actual values: e.g. sum all production_detail for production id = X and create corresponding inventory movements for each input with those quantities . This yields highly accurate inventory and allows spotting inefficiencies (like every time, actual butter used is 5

Without this, the system would deduct exactly recipe amounts from inventory for each production. That's simpler but doesn't capture waste/spill directly – differences would appear as unexplained inventory shrink over time, requiring occasional manual adjustments (which indeed many simpler systems do: they assume recipe usage, then do a stock count later and adjust differences as "waste").

In summary, ProductionDetail is not strictly required but is beneficial for advanced traceability and cost control.

Our design covers production record and implies inventory changes but leaves actual consumption tracking as either direct by recipe or via an optional detail table as needed.

9 Transfers Module

The Transfers module covers movement of stock between branches. It includes the **transfer** table for each transfer operation (with origin and desti-

nation branches, date, status, etc.) and **transfer_item** table for the details of products and quantities moved. This helps keep an audit of internal stock re-allocations (which do not affect overall company inventory but shift stock between locations).

9.1 Transfer (Traslado de stock)

The **transfer** table represents a transfer of items from one branch (origin) to another (destination). It records who sent it, who received it, when, and the status of the transfer.

Field	Data Type	Null?	Key / Constraints	Default
id_transfer	BIGINT	No	PK	(auto)
origin_branch_id	BIGINT	No	FK branch(id_branch)	→ -
dest_branch_id	BIGINT	No	FK branch(id_branch)	→ -
send_datetime	DATETIME	No		NOW()
receive_datetime	DATETIME	Yes		NULL
status	VARCHAR(20)	No		'SENT'
sent_by	BIGINT	No	FK user(id_user)	→ -
received_by	BIGINT	Yes	FK user(id_user)	→ NULL
notes	VARCHAR(255)	Yes		NULL

Field meanings:

- **id_transfer**: Primary key for the transfer record.
- **origin_branch_id**: The branch that is sending the stock (outgoing) .
- **dest_branch_id**: The branch that is receiving the stock (incoming) .
- **send_datetime**: Date/time when the transfer left origin (when it was shipped out) .
- **receive_datetime**: Date/time when the transfer was received at destination. NULL until actually received (if it's in transit) .
- **status**: Status of transfer. Possible values: "SENT" (or "IN TRANSIT") when dispatched, "RECEIVED" when completed, "CANCELLED" if aborted, etc. Default 'SENT' assuming it's recorded at send time .

Could use 'PENDING' for planned but not sent yet, depending workflow.

- **sent_by**: User who initiated/sent the transfer at origin (e.g. warehouse staff) . Required.
- **received_by**: User who confirmed receipt at destination . Null until received.
- **notes**: Optional comments, e.g. "Sent via van, ETA 5pm", "Check item count on arrival", or any anomalies (like "one box damaged") .

****Relationships:****

- **Transfer–Branch (Origin) (N–1)**: Many transfers originate from a branch .
- **Transfer–Branch (Destination) (N–1)**: Many transfers destined to a branch . (Note: origin and dest both reference branch table via separate FKs).
- **Transfer–User (Sent_by) (N–1)**: Many transfers sent by one user .
- **Transfer–User (Received_by) (N–1)**: Many transfers received by one user (e.g. receiving clerk) .
- **Transfer–TransferItem (1–N)**: A transfer has multiple item records listing products and quantities moved .
- **Transfer–InventoryMovement (1–N)**: Logically, each transfer causes:
 - Inventory movements at origin: negative for each item sent out (type "TRANSFER_OUT"), with reference to this transfer ID .
 - Inventory movements at destination: positive for each item received ("TRANSFER_IN"), ref this ID, when received .

This ensures stock leaves one and enters other, and we can reconcile that what left equals what arrived (or note differences).

9.2 Transfer Item (Detalle de Transferencia)

The **transfer_item** table lists each product and quantity being transferred in a transfer record.

Field	Data Type	Null?	Key / Constraints	Default
id_transfer_item	BIGINT	No	PK	(auto)
transfer_id	BIGINT	No	FK → transfer(id_transfer) ON DELETE CASCADE	–
product_id	BIGINT	No	FK → product(id_product)	–
quantity	DECIMAL(12,3)	No		0.000

****Field meanings:****

- **id_transfer_item**: PK.
- **transfer_id**: FK to the transfer record. Items cascade if transfer is deleted.
- **product_id**: The product being moved .
- **quantity**: Amount of product moved. Not null, default 0 (though 0 would not make sense logically). Can handle fractional if needed (e.g. transferring 2.5 kg of something).

****Relationships:****

- **TransferItem–Transfer (N–1)**: Many item rows per one transfer .
- **TransferItem–Product (N–1)**: Many transfers can involve the same product over time .

****Usage Scenarios:****

- **Stock Rebalancing Example**: Branch North is low on butter, Branch South has excess. They decide to send 5 kg from South to North. Process: - South's manager creates a transfer record: origin=South, dest=North, send_datetime now, status "SENT", sent_by = South manager. They add one transfer_item: product=Butter, quantity=5.0 kg . - Immediately, inventory at South is reduced by 5 kg (movement type TRANSFER_OUT) . The butter physically leaves South.

- The van drives to North. Later that day, North's manager opens this transfer record (which is likely visible as pending inbound) and records `receive.datetime` and `received.by` = North manager, status "RECEIVED". Now inventory at North is increased by 5 kg (movement `TRANSFER_IN`) . If any discrepancy (maybe only 4.5 kg arrived due to spillage), North's manager could adjust quantity in `transfer_item` or note it and possibly mark not fully received (depending on policy, but likely they'd update quantity to 4.5 and note missing 0.5).
 - The system can now verify that for transfer #77, origin out 5, dest in 5 (or 4.5 if not match). If not match, they might adjust inventory or create an adjustment for the loss of 0.5 on route (which might show as inventory shrink at some point, but might be simpler to note and have an adjustment entry).
 - The notes might record "Sent via ACME Couriers, box damage caused 0.5kg loss." .
- **Multi-item Transfer:** If transferring multiple products at once (say a whole selection of ingredients), they'd add multiple `transfer_items`. All are considered part of one shipment. Each item will generate separate movements for each product.
 - **In-Transit State and Auditing:** Until marked received, the stock is effectively in limbo. The origin branch's inventory already went down; destination hasn't gone up yet. If we want at any time to know "stock in transit", we could query transfers in status "SENT" and sum quantities. If one is stuck in transit too long, we can investigate.
 - Auditors can ensure that every `TRANSFER_OUT` movement has a corresponding `TRANSFER_IN`. The unique transfer ID ties those pairs . If a transfer is sent but not received (lost or very delayed), that's apparent either as open status or missing inbound movements.
 - Also, globally, transfers should not change total company stock: $\text{sum of all TRANSFER_OUT} = \text{sum of all TRANSFER_IN}$ (if none lost). The DB doesn't enforce that directly, but reports can check it.
 - **Cancelling a Transfer:** If a planned transfer is cancelled (maybe they decided not to send after creating record), one would update status to "CANCELLED". If it had already deducted origin inventory, one needs to add it back (maybe by deleting the transfer which cascades remove movements, but better to have a controlled process: e.g. mark cancelled and do inventory adjustment at origin for return of goods to shelf). Simpler if it was never physically sent, we could just reverse the movement or adjust origin stock. Our model doesn't ex-

explicitly handle cancellation beyond status and presumably manual fix of inventory if needed.

- **Transfers vs Sales/Purchases:** The transfer records are similar to a mini purchase order between branches. Odoo or ERP might treat them as internal picking orders. But importantly, transfers are internal: one branch's inventory loss is another's gain. The system ensures traceability but no financial transaction with external party. It's all internal cost shifting. Some advanced systems assign transfer costs (like internal pricing), but ours does not – it's purely quantity tracking (the Spanish mention "some solutions treat transfers as pseudo sale and purchase for costing if multi-company, but here one legal entity so it's neutral").
- **Stock Balance Report:** Using transfer data, one can generate a "stock ledger reconciliation". Eg. global formula: $\text{Sum}(\text{inventory all branches at end}) = \text{Sum}(\text{inventory at start}) + \text{Purchases} - \text{Sales} - \text{Wastes} + (\text{in} - \text{out transfers})$. Transfers cancel out in global sum but for branch-level, they appear accordingly . That helps detect shrink: if global doesn't balance, some unrecorded loss occurred.

With that, we've covered how the transfers are managed, allowing multi-branch operations to be coordinated .

10 Forecasting Module (Pronósticos de Demanda)

The Forecasting module involves the **sales_forecast** (PronosticoVenta) table which stores AI or analytically generated demand predictions for products at branches for future dates. This table is used to guide production and purchasing decisions by predicting how much of each product will likely sell. The telemetry and historical sales feed into these forecasts.

10.1 Sales Forecast (Pronóstico de Venta por IA)

The **sales_forecast** table holds demand predictions for a given product on a given future date (usually per branch), possibly with a confidence range and model version. Each entry might be one day's forecast for one product at one branch.

Field	Data Type	Null?	Key Con- straints	/	Default
<code>id_sales_forecast</code>	BIGINT	No	PK		(auto)
<code>branch_id</code>	BIGINT	No	FK	→	–
			branch(id_branch)		
<code>product_id</code>	BIGINT	No	FK	→	–
			prod- uct(id_product)		
<code>date</code>	DATE	No			–
<code>predicted_quantity</code>	DECIMAL(10,2)	No			0.00
<code>confidence_interval</code>	VARCHAR(50)	Yes			NULL
<code>generated_at</code>	DATETIME	No			NOW()
<code>model_version</code>	VARCHAR(20)	Yes			NULL

****Field meanings:****

- `id_sales_forecast`: PK.
- `branch_id`: Branch for which this prediction applies . Not null.
- `product_id`: Product being forecasted . Not null.
- `date`: The future date (or date period) for which the demand quantity is predicted . Likely a day.
- `predicted_quantity`: The forecasted sales quantity for that product on that date at that branch . (If forecasting weekly or monthly, could adjust meaning, but likely daily here).
- `confidence_interval`: Optional field capturing uncertainty or range, e.g. "±5" or "90"
- `generated_at`: Timestamp when this forecast was generated . Useful to know recency and to track if updated nightly. Not null.
- `model_version`: If the AI model or algorithm used has versions, this can record which version produced this prediction . Helps track improvements or compare versions (one can filter forecasts by version to evaluate model performance).

****Relationships:****

- **SalesForecast–Branch (N–1):** Many forecasts (for different products/dates) for one branch .
- **SalesForecast–Product (N–1):** Many forecasts for one product (over dates and branches) .
- (No direct link to sales, but one would later compare forecast vs actual sales by matching branch, product, date with actual sales sums).

****Usage Scenarios:****

- **Nightly Forecast Generation:** Each night, an AI system runs using recent sales, weather, day-of-week, etc. to predict next few days' demand . For example, on Aug 15, it predicts for Aug 16: Branch Center, product Bolillo, predicted_quantity=120.0, confidence "±10", generated_at Aug 15 23:00, model_version "v2.3". It inserts a row. It does similarly for all key products and branches, perhaps for a 3-day horizon .
- **Using Forecast for Production Planning:** On Aug 16 morning, the manager sees tomorrow (Aug 17) forecast says e.g. 80 bolillos for branch Center . They decide to produce a bit more as buffer, say 90 . They create a production plan accordingly. They might note actual produced vs forecast (130 vs 120 in our example) for evaluation.
- **Forecast vs Actual Comparison:** After the day, the AI team or system can evaluate how forecasts did: e.g. predicted 120, actually sold 125 (so error +4.2
- **Feeding into Reordering:** If forecast says next 3 days need 500 kg flour, the purchase system might suggest ordering if current stock + on-order - usage > threshold. In more advanced use, one might use forecasts in an MRP (material requirements planning) for ingredients. Our schema doesn't directly connect forecast to purchase, but managers use it manually or via separate logic.
- **Multi-branch rebalancing suggestion:** The forecast could even suggest transfers: e.g. one forecast scenario in Spanish: if branch A will sell more than stock, and branch B has excess, schedule a transfer the night before . This isn't automatic in schema, but user can interpret forecast combined with inventory to decide transfers. Possibly future system might auto-create transfer records based on forecast, but that's beyond our scope.

- **Telemetry Integration:** Forecast model likely uses telemetry data (like temperature, humidity or foot traffic if available) as input. Those readings are stored in SensorReading table (Telemetry module), but the forecast generation code (external to DB perhaps) would query that. We do not necessarily store direct link, but one could note in confidence or version if a particular scenario was considered.
- **Glossary Terms:** "Pronóstico" means forecast, our table covers that. Confidence interval usage example: "90"
- **Maintaining History:** Each forecast run generates records. If we do daily forecasts for next 3 days, everyday new rows for overlapping dates could be created (with new generated_at and possibly new version). One could either update previous forecasts or insert new ones. It's likely better to insert new ones with perhaps updated model_version. If multiple forecasts exist for same branch/product/date, we have to decide which is current. We didn't set a uniqueness constraint on (branch, product, date) – perhaps we should to avoid duplicates. We could add unique (branch, product, date, model_version) or if only one model used at a time, unique on (branch, product, date) and update daily. But tracking old forecasts is useful for model evaluation. Possibly keep them (so no unique, but then to get latest one, filter by max generated_at). It's a design decision – here, likely we allow multiple but with model_version we can differentiate scenarios. Real system might store all, or store only latest forecast for each day (overwriting prior). We follow storing all for traceability.

Overall, the Forecast table helps close the loop: data (sales, telemetry) -> forecast -> decisions (production, transfers, purchases) -> actual outcomes -> back to data . This creates an optimization cycle to reduce waste and improve service.

11 Telemetry Module

The Telemetry module captures data from IoT sensors installed in the bakery (e.g. temperature/humidity sensors in ovens or fridges, foot traffic counters, etc.) and possibly any external data. It includes the **sensor** table to identify each device and the **sensor_reading** table to store periodic measurements. Such data can feed analytics like forecasting or quality monitoring.

11.1 Sensor (Dispositivo IoT)

The **sensor** table lists each IoT sensor device deployed, with its type and location, and keeps track of its status.

(Already described in the earlier Administration/Telemetry integrated explanation, but summarizing key fields again for clarity.)

****Key fields (from earlier “Sensor” section)**:**

- **id_sensor**: PK for sensor.
- **branch_id**: Branch where installed .
- **name**: Descriptive name (e.g. "Oven1 Temp") .
- **type**: Type of sensor/metric (e.g. TEMPERATURE, HUMIDITY, PEOPLE_COUNT) .
- **location**: Physical location description (e.g. "Front Oven #1", "Refrigerator", "Store Entrance") .
- **description**: Detailed description (model, etc.) .
- **is_active**: 1 if sensor is currently active, 0 if removed or broken .
- **installation_date**: When installed .

Relationships:

- Sensor–Branch: N–1 (branch has many sensors) .
- Sensor–SensorReading: 1–N (sensor produces many readings) .

Usage:

- Identify sensors in branch for UI (list all active sensors to display readings).
- Provide metadata for readings (know what sensor1 means).
- If threshold alerts are needed, could add fields (min_acceptable, max_acceptable as mentioned) to sensor config for alert generation.
- Example: sensor TempHorn01 (Temperature Oven1) at Central, type=TEMPERATURE, installed on date, active=1. When reading goes above threshold (say > 250°C), the app could trigger an alert record (not in schema, but procedure).

11.2 Sensor Reading (Lectura de Sensor)

The **sensor_reading** table logs each measurement from sensors, with timestamp and value.

Key fields:

- **id_sensor_reading**: PK.
- **sensor_id**: FK to sensor (which sensor produced this) .
- **measured_at** (we named timestamp as measured_at or date_time): The timestamp of reading .
- **value**: The numeric reading value (to 3 decimals maybe) .
- **unit**: Unit of the reading (°C,
- **processed**: A boolean (BIT) indicating if this reading has been processed by analytics (like integrated into forecasts or anomaly detection) . Could default 0 (not processed), then set 1 after analysis to avoid re-processing duplicates.

Relationships:

- SensorReading–Sensor: N–1 (reading belongs to a sensor) .

Usage scenarios:

- A temperature sensor posts reading every minute: these get stored with timestamp and value. We can query last known temperature for display (dashboard) or average daily oven temp etc.
- The AI forecast might use external temperature: e.g. if it's hot outside, more cold drinks sales. If an external weather sensor is in the system (or we import it as a sensor reading type EXTERNAL_TEMP with branch maybe as city or special branch).
- Anomalies: If a fridge temperature reading exceeds threshold (set in sensor), system generates alert (not in schema, but concept) .
- Data retention: sensor_reading can grow huge. Partitioning by date is advised (like monthly tables), or at least indexing by sensor+time for range queries . The indexing/partition strategy section covers that.

- Example: sensor_id=1 (Central Oven1 Temp), measured_at "2025-08-16 05:00", value=180.5, unit="°C", processed=0. Many such per day. Forecast model might pick up average oven temp for early morning hours to correlate with bread quality or output.
- Another example: sensor type PeopleCounter at entrance: records count of entries per hour. Those readings can correlate with sales (foot traffic).
- The processed field can be used by ETL to mark readings that have been aggregated into some higher-level feature, to not double-count if repeating ETL.

Thus, telemetry data stored here closes the loop by feeding into forecasting or quality control, and combined with sales and production data can yield deeper insights (like conditions that affect sales or product shelf life, etc.) .

A Glossary of Terms

(English – Spanish key terms as they appear in the database context)

- **Branch** – Sucursal: A physical store or bakery location .
- **Product** – Producto: An item sold or used (either finished good or raw material) .
- **Raw Material / Input** – Insumo: Ingredient or input product used in production (not sold directly) .
- **Recipe** – Receta: Formula detailing ingredients to produce a product .
- **Production Batch / Lot** – Lote de Producción: A recorded instance of producing a quantity of product .
- **Sale** – Venta: A customer purchase transaction .
- **Sale Item** – Detalle de Venta: A line item in a sale (specific product sold) .
- **Cash Session** – Sesión de Caja: Register opening/closing period for a cashier .

- **Purchase** – Compra: An order/receipt of goods from a supplier .
- **Purchase Item** – Detalle de Compra: A line item in a purchase (specific product ordered) .
- **Supplier** – Proveedor: A vendor supplying goods to the bakery .
- **Customer** – Cliente: A registered customer or company (for loyalty or invoicing) .
- **Transfer** – Transferencia: Stock transfer between branches .
- **Forecast** – Pronóstico: A demand prediction for future sales .
- **Telemetry** – Telemetría: Sensor-collected data (temperature, humidity, etc.) .
- **Sensor** – Sensor (Dispositivo IoT): An IoT device measuring something (temperature, humidity, count) .
- **Reading (Sensor)** – Lectura (de Sensor): A single data measurement from a sensor .
- **Active** – Activo: (For branch, user, etc.) Indicates it is in service (not deleted) .
- **Inventory** – Inventario (Existencias): Stock on hand of products at a branch .
- **Inventory Movement** – Movimiento de Inventario: A log entry of stock change (increase or decrease) .
- **POS** – Punto de Venta: Point of Sale system/module (sales and cash register).
- **AI** – IA (Inteligencia Artificial): Artificial Intelligence (used in forecasting context).
- **PITR** – Producción, Inventario, Transferencias, Reposición (just to recall context of integrated operations).

B Integrity Constraints and Validation Rules

The database schema employs numerous integrity constraints and implied validation rules to maintain data consistency:

- **Primary Keys:** Every table has an ‘id’ primary key that uniquely identifies rows. Most are auto-increment integers (BIGINT) for uniformity and capacity.
- **Foreign Keys and Referential Integrity:**
 - Branch references appear in user, inventory, sale, purchase, production, transfer, etc., ensuring those records cannot exist without a valid branch. For example, ‘user.branch_id’ must match an existing branch or be NULL. All FKs are set to RESTRICT or CASCADE appropriately to prevent orphan records (e.g., cannot delete a branch if any users or records reference it; cannot delete a product if inventory or sales exist for it).
 - ‘sale.customer_id’ links to customer, allowing NULL for walk-ins .
 - Self-referencing FKs like ‘user.created_by -> user.id_user’ ensure a user record’s creator exists; we might use ON DELETE SET NULL or PROTECT (so that if the creating user is removed, we preserve or nullify that link) .
 - ‘transfer.origin_branch_id’ and ‘dest_branch_id’ are separate FKs to branch table; a check or logic ensures they are not equal (should not transfer from a branch to itself – we note this in logic but could enforce via CHECK if desired) .
 - All FKs are designed to maintain referential coherence, e.g. cannot have an inventory record pointing to a non-existent product or branch .
- **Unique Constraints:**
 - Certain fields that serve as alternate IDs or require uniqueness:
 - Branch name is unique to avoid duplicates .
 - Category name unique .
 - Role code unique.
 - User email unique (each user login must be unique) .
 - Product SKU unique (ensures no two products share a SKU).
 - Inventory composite unique on (branch_id, product_id) so one stock record per product per branch .
 - Recipe

product_id unique (one recipe per product at a time) . - Possibly forecast could have unique (branch, product, date, model_version) or at least (branch, product, date) if we only keep latest. We did not define it strictly, but logically, one forecast per product per day per version.

- **Not Null and Default Rules:**

- Fields that are essential have NOT NULL: e.g., product name, category in product, user role, branch in user, etc. This prevents incomplete records (ensuring required data is present).
- Boolean flags default to TRUE for active fields (meaning new entries are active by default) .
- Numeric fields often default to 0 (e.g. inventory.quantity default 0, reserved 0 , sale_payment amount 0) to avoid null confusion. But logically, these are then updated accordingly.
- Timestamps default to NOW() for created_at fields so audit is automatically filled without relying on application to set it.
- Dates like issue_date, send_datetime default to current so that record has a meaningful timestamp by default (though ideally application sets actual date).

- **Check Constraints:**

- Ensure no negative stock or negative reserved: ‘inventory.quantity $\geq$ 0’, ‘reserved $\geq$ 0’ .
- Ensure positive values where expected: ‘sale_item.quantity $\geq$ 0’ (implicitly by usage, or could be CHECK $\geq$ 0 if fractions allowed maybe $\geq$ 0.001, we didn’t add but assume no zero/negative quantity sales), ‘purchase_item.quantity_ordered $\geq$ 0’, ‘quantity_received $\geq$ 0’ (if none received, could be 0 or NULL, but if partially, then 0 allowed to indicate nothing came yet? We left it as nullable anyway).
- ‘inventory_movement.quantity_change’ doesn’t have an explicit CHECK but by convention a positive or negative is meaningful. If an erroneous 0 movement was inserted, it would do nothing – likely we trust application not to insert 0 (or we could enforce quantity_change $\neq$ 0).

- ‘movement_type’ – we didn’t implement an ENUM in SQL directly, but an application should only set allowed values. If using Postgres, we might create an ENUM type for movement_type and use that . Similarly, ‘payment.method’ ideally limited to known types, ‘status’ fields limited to known states. Those might be enforced via CHECK IN (‘value1’,‘value2’,...) if needed. We rely on controlled inputs.
- We might enforce that a transfer origin and destination must be different (application logic or CHECK: origin_id $\neq$ dest_id) .
- If we had something like a user cannot be their own creator (not necessary, no strong reason to forbid, and admin likely creates all including themselves by bootstrap).
- If production might enforce ‘quantity_produced $\geq$ 0’ (you wouldn’t record 0 production).
- Forecast likely ‘predicted_quantity $\geq$ 0’ (cannot predict negative sales).

- **Default Cascade Actions:**

- If a sale is deleted (e.g., a test record or cancellation), its sale_items and sale_payments cascade delete, cleaning all child records .
- If a purchase is deleted, its purchase_items cascade delete.
- If a transfer is deleted, its transfer_items cascade, and you’d manually have to handle inventory since we typically wouldn’t delete a transfer after processing (except if mistake, then one should manually correct inventory or simply not allow deletion once processed).
- We protected some relationships instead: e.g. if trying to delete a product that still has sales or inventory, the DB will throw an error preventing it (due to those FKs, by default restrict if not specified cascade or set null).
- User references (like created_by in many tables) – we might set those FKs as ON DELETE SET NULL, meaning if a user is deleted, we don’t want to delete all branches they created etc., but we null out that audit reference . Similarly for updated_by. This keeps history but allows removing an account if needed (though often better to just mark user inactive rather than delete).

- **Application-level validations:**

- Uniqueness of names (e.g. branch names) might also be enforced by UI when adding.
- Formats: e.g. User email should be a proper email format (the DB can't easily enforce that beyond maybe a regex check if desired). Mentioned in Spanish: "Debe validarse formato de email" – that's an application validation.
- Phone numbers format can be checked (digits, length) by application.
- Product name not empty, etc. (already not null covers it, but also maybe no special characters).
- Ensuring logically consistent data:
 - If `sale.status = "CANCELLED"`, ideally `sale.items` and payments should either not exist or be zeroed out. We rely on procedure or triggers if we wanted to enforce (not detailed).
 - If `purchase.status = "RECEIVED"`, then all `purchase_item.quantity_received` should be filled (not null). Perhaps if any is null and status is received, that's inconsistent.
 - If `transfer.status = "RECEIVED"`, then `receive_datetime` and `received_by` should be not null, and an application should ensure that. Could enforce by triggers or by splitting status into separate boolean, but current design expects application to handle it.
 - `inventory_movement`: The combination (`branch_id`, `product_id`, `timestamp`) not unique but chronological ordering by timestamp is assumed. If two movements have identical timestamps order is undefined – not a big problem if small variation. Could ensure auto-order by sequence if needed.
- Cross-table logic:
 - Sum of `sale_payments` amounts = `sale.total_amount` for each sale. We don't enforce in DB, but logically every sale should be fully paid (or if on credit, method "ACCOUNT" covers full, etc.). Application should ensure the sum matches before closing a sale.
 - Sum of `purchase_items` (ordered or received) * cost = `purchase.total_amount`. We somewhat maintain that by storing totals and updating them, but not automatic. Could use triggers to update purchase total when items update.
 - For inventory: whenever a sale/purchase/production happens, application triggers appropriate inventory movements and updates `inventory.quantity`. Integrity wise, inventory is a derived state from movements. Some systems not even store inventory table, just compute from movements. We store for quick access but

must keep it in sync with movements. That is an invariant: `inventory.quantity` = sum of all movements up to date for that branch/product (taking initial as 0). The DB alone doesn't enforce that – it relies on every transaction logging a movement and updating inventory. If something fails and they diverge, an inventory audit script might be needed to recalc and fix differences.

- Audit fields: e.g. `created_at` should be $\leq$ `updated_at` if both exist – application should ensure not to set `updated_at` earlier than `created_at` logically. Not enforced but naturally likely fine (unless system clock issues).

These constraints collectively guarantee foreign key consistency (no orphan references), uniqueness where needed (no duplicates of key entities), non-negativity of critical counts and amounts, and proper deletion or retention of records for audit. Additional validations (formats, business logic that one table's data matches another's) are handled at the application or via triggers if complexity is needed, but the design as outlined provides a solid base of integrity .

C Indexing and Partitioning Strategy

To ensure good performance as data grows, we establish indexes on key fields, and consider partitioning certain large tables by date. The following indexing strategy is applied (explicitly or implicitly by foreign keys/PKs):

- **Primary Key Indexes:** All PKs automatically create indexes on the id columns (e.g., `id_product`, `id_sale`, etc.). These allow fast lookup by primary key.
- **Foreign Key Indexes:** We index all foreign key columns for efficient joins and filtering :
 - `user(branch_id)` and `user(role_id)`: to quickly find users by branch or role (e.g. list users in a branch) .
 - `product(category_id)`: to get products per category quickly .
 - `inventory(branch_id, product_id)`: composite index (and uniqueness) already defined . This helps look up stock of a product in a specific branch, or get all products' stock in a branch. We have a unique composite which doubles as an index for queries on branch and product.

- `inventory_movement(branch_id, occurred_at)` (or timestamp): a composite index on branch and timestamp for time-series queries per branch . Also `inventory_movement(product_id, occurred_at)` if we often query movements of a specific product across branches by time (we might even do both composite or separate indexes on product and branch and on date). Provided in code as examples (like `ix_move_branch_time`, `ix_move_product_time`).
- `inventory_movement(ref_type, ref_id)`: index to quickly find all movements related to a specific document (e.g. all movements for transfer id X, or purchase id Y) . This helps verifying that for each transfer all items accounted (we have `ix_move_ref` in code).
- `sale(branch_id)`: if one often queries sales by branch (e.g. daily sales per store).
- `sale(datetime)`: indexing sale date can help daily report queries (or composite (branch, date)). If queries often filter by date range and branch, a composite (branch_id, date) is beneficial, or just indexing datetime if lots of chronological queries (the sale PK itself might be chronological if it's an identity, but not guaranteed).
- `sale(customer_id)`: to find all sales for a customer (reporting loyalty or AR) .
- `sale_item(product_id)`: to aggregate total sales of a product quickly (though we might join sale and filter by date too). Possibly composite index on (product_id, sale_id) – but sale_id is PK in sale_item anyway (since sale_id not unique but sale_item PK is separate).
- `sale_payment(method)`: If one frequently sums by method, an index on method might help though it has few categories – better to index (sale_id, method) for queries per sale if needed. However, usually one queries sale_payment by sale (which will be via sale_id which is indexed as it's an FK likely auto-indexed).
- `purchase(branch_id)` and `purchase(supplier_id)`: to filter POs by branch or supplier (e.g. total by supplier queries) .
- `purchase(datetime or issue_date)`: for date-range queries on purchases.
- `purchase_item(product_id)`: to find where a product was purchased (if analyzing supplier usage of a product).
- `transfer(origin_branch_id)` and `transfer(dest_branch_id)`: to query transfers out of or into a branch.

- `transfer(status)`: maybe index status if often searching for pending transfers to receive.
 - `transfer_item(product_id)`: if one wants to see all transfers involving a product (maybe not common, but if investigating shrink might see how often product moved).
 - `sensor(branch_id)`: to list sensors per branch (some UI listing).
 - `sensor(type)`: if wanting to group sensors by type (less critical).
 - `sensor_reading(sensor_id, measured_at)`: crucial index, since this table will be large; composite index allows retrieving all readings of a sensor in time order quickly .
 - `sensor_reading(measured_at)`: if one does global time queries (not likely without filtering by sensor).
 - `sales_forecast(branch_id, date)` and `sales_forecast(product_id, date)`: to retrieve forecasts by branch/date or product/date quickly (like all forecasts for tomorrow, or all forecasts for product X across branches).
- **Composite Index for Inventory unique:** We defined `unique(branch, product)` which is effectively also an index . Also we explicitly listed a composite index on `inventory_movement` for `branch+date` and `product+date`, as well as an index on `(ref_type, ref_id)` for referencing docs . These serve performance and ensure data consistency logically (the unique ones).
 - **Partitioning (Optional at Scale):** For very high volume tables, we consider partitioning by date (monthly or yearly) to keep old data manageable :
 - `inventory_movement`: This grows with every sale, purchase, etc. We could partition by year or month. For example, a separate partition per month, so queries for recent movements hit a smaller chunk, and archiving old ones is easier .
 - `sale_item` and `sale_payment`: These could be partitioned by year as well (especially if millions of sales per year). Usually, `sale` and `sale_item` are manageable if business isn't huge, but if needed, partition `sale` by `sale_date` (year) and have `sale_item` inherit that partitioning via `sale_id` references. This complicates some querying but can help manage indexes.

- **sensor_reading:** This can be extremely high frequency (multiple sensors, every few seconds). Partitioning by date (e.g. by day or month) is highly recommended so that old data can be dropped or summarized. We might keep 1 year of raw readings then purge or aggregate older, for example. Partition by month is typical, each partition holds one month of readings, making queries on recent months faster and maintenance easier.
- **sales_forecast:** Probably doesn't need partition since forecasts maybe only stored for short horizon or overwritten. But if we keep many versions, possibly partition by generated_at year just to keep old ones separate. However, forecast table likely stays smaller (it's only a few days ahead for each product/branch).

Partitioning usage: e.g. in PostgreSQL, one could do 'CREATE TABLE inventory_movement_y2025 PARTITION OF inventory_movement FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');' etc. This allows dropping a whole partition when that year's movements are no longer needed (or archived).

- **Additional Performance Considerations:**

- Use of covering indexes or index include: e.g., for frequent query "sum total_netto by branch and date", one might create an index on sale(branch_id, date, total_amount) to allow index-only retrieval without hitting table.
- Full Text indexes: not needed given our data (maybe for product name search, a simple index on name suffices or using LIKE).
- Transaction volumes: Inventory movements and sensor readings are likely highest volumes. We indexed them for branch/time etc. Partitioning them is key as noted. Others like sales, purchase, production are lower volume comparatively (depending on business size).
- If multi-branch queries on inventory often need to sum all branches, having inventory table unpartitioned is fine (it's not terribly large because it's one record per product per branch).
- The composite index on (branch_id, product_id) in inventory helps both branch filter and product filter. If we often query "give me all branches stock of product X", that index can also be used by searching on product_id (since branch_id is first in composite, it's

not ideal for searching by product alone unless using index skip scan or we could add a separate index on `product_id` if needed. Considering usage: probably more common to query inventory by branch, but cross-branch comparison might also happen, so maybe add index on `product_id` too).

- On movement table, separate index on `product_id` alone might also help "all movements of product X across all branches/time" for product trace (though we included composite with `product + time`, which still helps if scanning by product with time sorted).
- Forecast table indexing `branch+date` covers retrieving all forecasts for a day (across products) to plan production, and `product+date` covers focusing on one product's forecast across branches (less likely scenario, but maybe to see total demand for that product across chain on a date).

In summary, we index foreign keys and key query columns to speed up joins and filters. For huge tables, we partition by date (monthly/yearly) to keep data manageable and pruning easy. This ensures the system scales: e.g., sensor readings millions of rows can be partitioned so queries for recent days don't slog through years of data. Similarly, inventory movement and sale item logs partitioned by year avoid index bloat and make historical queries (or deletion after X years) easier. These measures align with best practices for high-volume systems.

D Sample SQL DDL Fragments

Below are some example SQL Data Definition Language snippets illustrating how parts of this schema can be created in PostgreSQL. This includes table creation with key constraints and indexes. (For brevity, not all tables are shown, but the pattern is similar.)

— *Example: Branch table*

```
CREATE TABLE branch (  
    id_branch BIGSERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL UNIQUE,  
    address VARCHAR(255),  
    phone VARCHAR(20),  
    email VARCHAR(100),  
    city VARCHAR(50),  
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
```

```

        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        created_by BIGINT NOT NULL REFERENCES "user"(id_user) ON DELETE RESTRICT
    );
CREATE INDEX ix_branch_active ON branch(is_active);

```

— *Example: Inventory table*

```

CREATE TABLE inventory (
    id_inventory BIGSERIAL PRIMARY KEY,
    branch_id BIGINT NOT NULL REFERENCES branch(id_branch) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES product(id_product) ON DELETE CASCADE,
    quantity NUMERIC(14,3) NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    reserved NUMERIC(14,3) NOT NULL DEFAULT 0 CHECK (reserved >= 0),
    reorder_point NUMERIC(14,3),
    last_updated TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uq_inventory UNIQUE(branch_id, product_id)
);
CREATE INDEX ix_inventory_branch_product ON inventory(branch_id, product_id);

```

— *Example: Inventory Movement table*

```

CREATE TYPE movement_type AS ENUM ( 'SALE', 'PURCHASE', 'PRODUCTION_CONSUMPTION',
    'PRODUCTION_OUTPUT', 'TRANSFER_OUT', 'TRANSFER_IN', 'ADJUST_POS', 'ADJUST_NEG' );
CREATE TABLE inventory_movement (
    id_movement BIGSERIAL PRIMARY KEY,
    branch_id BIGINT NOT NULL REFERENCES branch(id_branch) ON DELETE RESTRICT,
    product_id BIGINT NOT NULL REFERENCES product(id_product) ON DELETE RESTRICT,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    movement_type movement_type NOT NULL,
    quantity NUMERIC(14,3) NOT NULL,
    ref_type VARCHAR(20),
    ref_id BIGINT,
    user_id BIGINT NOT NULL REFERENCES "user"(id_user),
    note VARCHAR(255)
);
CREATE INDEX ix_move_branch_time ON inventory_movement(branch_id, occurred_at);
CREATE INDEX ix_move_product_time ON inventory_movement(product_id, occurred_at);
CREATE INDEX ix_move_ref ON inventory_movement(ref_type, ref_id);

```

— *Example: Sale and Sale_Item tables*

```

CREATE TABLE sale (
    id_sale BIGSERIAL PRIMARY KEY,

```

```

        branch_id BIGINT NOT NULL REFERENCES branch(id_branch),
        cashier_id BIGINT NOT NULL REFERENCES "user"(id_user),
        session_id BIGINT REFERENCES cash_session(id_cash_session),
        customer_id BIGINT REFERENCES customer(id_customer),
        datetime TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        status VARCHAR(20) NOT NULL DEFAULT 'COMPLETED',
        total_amount NUMERIC(10,2) NOT NULL,
        notes VARCHAR(255)
    );
CREATE INDEX ix_sale_date ON sale(datetime);
CREATE INDEX ix_sale_branch ON sale(branch_id);
CREATE INDEX ix_sale_customer ON sale(customer_id);

CREATE TABLE sale_item (
    id_sale_item BIGSERIAL PRIMARY KEY,
    sale_id BIGINT NOT NULL REFERENCES sale(id_sale) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES product(id_product),
    quantity NUMERIC(10,3) NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10,2) NOT NULL,
    total_line NUMERIC(10,2) NOT NULL
);
CREATE INDEX ix_sale_item_product ON sale_item(product_id);

```

(The code above demonstrates creation of branch, inventory, inventory_movement, sale, and sale_item tables with constraints and indexes. Other tables such as product, user, purchase, etc. would follow similarly, defining FKs and necessary constraints as described. The actual schema in a real deployment would include all tables.)

These DDL statements reflect how the design can be implemented in SQL. For example, the `inventory_movement` table uses a PostgreSQL ENUM type for movement types for clarity and to prevent invalid values. We also show explicit index creation; note that PostgreSQL automatically indexes primary keys and unique constraints, and often foreign keys (if not auto-indexed, we created them as needed).

This sample DDL can be expanded to all tables, ensuring that:

- Appropriate `REFERENCES` clauses enforce foreign keys.
- `ON DELETE` rules are applied (`CASCADE` where child data should go, `RESTRICT/NO ACTION` where we want to prevent deletion if in use, or `SET NULL` for audit fields).

- Constraints like CHECKs are included (as demonstrated for non-negatives, positive quantities, etc.).
- Indexes are created to support frequent query patterns (some shown above, e.g., `ix_move_ref` to quickly find movements by reference document) .

Compiling the full set of DDL for all tables would produce a SQL script that can be run on a PostgreSQL database to instantiate the ZeroMerma operational database schema, ready for use in managing the multi-branch bakery's data. Each element of the schema together supports enforcement of data integrity and efficient querying for day-to-day operations and analysis .